

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Nguyễn Tiến Đạt

XÂY DỰNG HỆ THỐNG SSO PHI MÁY CHỦ

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Công nghệ thông tin

Hà Nội – 2023

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**

Nguyễn Tiến Đạt

XÂY DỰNG HỆ THỐNG SSO PHI MÁY CHỦ

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Công nghệ thông tin

Cán bộ hướng dẫn: PGS. TS. Trương Anh Hoàng

Cán bộ đồng hướng dẫn: ThS. Nguyễn Quang Minh

Hà Nội – 2023

LỜI CẢM ƠN

Đầu tiên em xin gửi lời cảm ơn chân thành đến PGS. TS. Trương Anh Hoàng và ThS. Nguyễn Quang Minh đã đồng hành, tận tình hướng dẫn và đưa ra những góp ý cho em trong thời gian làm khóa luận vừa qua.

Em cũng xin bày tỏ lòng biết ơn chân thành đối với toàn bộ các thầy, cô giáo và cán bộ của Trường Đại học Công nghệ nói chung, cũng như các thầy, cô giáo của Khoa Công nghệ thông tin nói riêng. Những kiến thức, kinh nghiệm và sự hướng dẫn tận tình trong suốt bốn năm vừa qua của các thầy, cô sẽ là hành trang vững chắc giúp em thành công trong cuộc sống sau này.

Tôi cũng xin gửi lời cảm ơn đến các bạn sinh viên của Trường Đại học Công nghệ, nhất là các bạn sinh viên K64CD đã luôn đồng hành, giúp đỡ tôi trong suốt bốn năm học đại học.

Cuối cùng, con xin gửi tới bố, mẹ cùng gia đình lòng biết ơn và tình cảm yêu thương chân thành nhất vì đã luôn ủng hộ, động viên con trong suốt những năm qua, là hậu phương vững chắc giúp con vượt qua những khó khăn và thử thách trong suốt khoảng thời gian vừa qua.

Hà Nội, ngày 25 tháng 04 năm 2023

Nguyễn Tiến Đạt

TÓM TẮT

Tóm tắt: Ngày nay, với sự phát triển của các hệ thống thông tin, đặc biệt là Internet, người dùng có thể tiếp cận và truy cập vào rất nhiều các ứng dụng, dịch vụ khác nhau. Một trong những khía cạnh quan trọng của rất nhiều các ứng dụng, dịch vụ đó là yêu cầu về việc xác thực cũng như bảo mật và quản trị danh người dùng. Một trong những cách phổ biến được các hệ thống sử dụng đó là xác thực người dùng bằng tài khoản/mật khẩu. Tuy vậy, chính bởi sự bùng nổ của các sản phẩm phần mềm, số lượng tài khoản/mật khẩu mà một người dùng cần phải tạo và ghi nhớ là vô cùng lớn. Từ đó, các giải pháp xác thực SSO (Single Sign-On) được cung cấp bởi các bên thứ ba (như Google, Microsoft, Facebook,...) xuất hiện nhằm giải quyết vấn đề trên. Hiện nay, một số doanh nghiệp có nhiều hệ thống nội bộ với các chính sách an ninh thông tin đặc thù khiến họ không thể sử dụng các dịch vụ SSO được cung cấp bởi bên thứ ba, đòi hỏi các doanh nghiệp này phải tự phát triển các hệ thống SSO của riêng mình. Tuy nhiên, việc tự triển khai và vận hành các hệ thống SSO đòi hỏi nhiều công sức cũng như những chi phí rất lớn trong quá trình phát triển và vận hành hệ thống. Một giải pháp có thể được sử dụng đó là đưa hệ thống SSO của doanh nghiệp lên đám mây và tận dụng những lợi thế của các dịch vụ “phi máy chủ” (serverless). Khóa luận này sẽ tập trung giới thiệu giải pháp xây dựng một **Hệ thống SSO phi máy chủ** bao gồm các yêu cầu của hệ thống, các công nghệ sử dụng để xây dựng, phát triển cũng như quản lý tài nguyên trên đám mây một cách hiệu quả.

Từ khóa: *Hệ thống SSO, Single Sign-On, phi máy chủ.*

LỜI CAM ĐOAN

Tôi xin cam đoan khóa luận tốt nghiệp **“Xây dựng Hệ thống SSO phi máy chủ”** là do tôi tự tìm hiểu, nghiên cứu và trình bày dưới sự hướng dẫn của PGS. TS. Trương Anh Hoàng cùng ThS. Nguyễn Quang Minh, không có sự sao chép công trình nghiên cứu của người khác và cũng chưa từng được nộp như một báo cáo khóa luận tại Trường Đại học Công nghệ - Đại học Quốc gia Hà Nội hoặc bất kỳ trường đại học khác. Những gì tôi viết ra trong khóa luận này không sử dụng các kết quả nghiên cứu của người khác mà không trích dẫn cụ thể. Tôi cam kết những nội dung tham khảo trong khóa luận nằm trong giới hạn, phạm vi cho phép theo quy chế của trường. Tôi cũng xin cam đoan những nội dung, mã nguồn tôi trình bày trong khóa luận là do tôi tự phát triển. Nếu không đúng sự thật, tôi xin hoàn toàn chịu trách nhiệm theo quy định của Trường Đại học Công nghệ - Đại học Quốc gia Hà Nội.

Tác giả khóa luận

Nguyễn Tiến Đạt

MỤC LỤC

LỜI CẢM ƠN	i
TÓM TẮT	ii
LỜI CAM ĐOAN	iii
MỤC LỤC	iv
DANH MỤC HÌNH VẼ	vii
DANH MỤC BẢNG BIỂU	ix
DANH MỤC CỤM TỪ VIẾT TẮT	x
MỞ ĐẦU	1
CHƯƠNG 1. CƠ SỞ LÝ THUYẾT	3
1.1. Single Sign-On.....	3
1.1.1. Giới thiệu chung về Single Sign-On.....	3
1.1.2. Một số giải pháp Single Sign-On	4
1.2. Giải pháp Single Sign-On phi máy chủ	5
1.2.1. Giới thiệu tổng quan	5
1.2.2. Điểm mạnh của hệ thống SSO phi máy chủ.....	7
1.3. Điện toán phi máy chủ	7
1.4. Amazon Cognito	8
1.5. AWS Lambda.....	8
1.6. Amazon API Gateway	9
1.7. Amazon S3.....	10
1.8. Amazon CloudFront.....	10
1.9. AWS WAF	11
1.10. Amazon DynamoDB.....	11
1.11. Hạ tầng dưới dạng mã IaC	12
1.11.1. Tổng quan về IaC	12
1.11.2. Terraform.....	13
1.11.3. Terragrunt	14
1.12. ReactJS.....	15

1.13. Tổng kết chương	15
CHƯƠNG 2. PHÂN TÍCH YÊU CẦU	16
2.1. Xác định bài toán và đối tượng sử dụng	16
2.2. Phân tích các yêu cầu của hệ thống	16
2.2.1. Yêu cầu chức năng thêm tài khoản mới và cập nhật thông tin.....	16
2.2.2. Yêu cầu chức năng phân quyền vào nhóm nghiệp vụ	17
2.2.3. Yêu cầu chức năng đăng nhập một lần.....	17
2.2.4. Yêu cầu chức năng đặt/đặt lại mật khẩu.....	17
2.2.5. Các yêu cầu phi chức năng	18
2.3. Phân tích và đặc tả ca sử dụng	18
2.3.1. Đặc tả ca sử dụng Đăng nhập	19
2.3.2. Đặc tả ca sử dụng Tạo tài khoản.....	20
2.3.3. Đặc tả ca sử dụng Thiết lập thông tin đăng nhập	21
2.3.4. Đặc tả ca sử dụng Quản lý tài khoản	22
2.3.5. Đặc tả ca sử dụng Quản lý nhóm phân quyền	23
2.3.6. Đặc tả ca sử dụng Đăng xuất	24
2.4. Tổng kết chương	24
CHƯƠNG 3. THIẾT KẾ HỆ THỐNG	25
3.1. Tổng quan thiết kế hệ thống.....	25
3.2. Mô tả luồng tổng quan của hệ thống.....	27
3.3. Thiết kế tầng dữ liệu	29
3.4. Thiết kế luồng Đăng nhập.....	34
3.5. Thiết kế luồng Tạo tài khoản	35
3.6. Thiết kế luồng Thiết lập thông tin đăng nhập	36
3.7. Thiết kế luồng Quản lý tài khoản.....	37
3.8. Thiết kế luồng Quản lý nhóm phân quyền.....	39
3.9. Thiết kế luồng Đăng xuất.....	41
3.10. Tổng kết chương	42

CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG, CÀI ĐẶT VÀ KIỂM THỬ	43
4.1. Xây dựng cấu trúc mã nguồn	43
4.1.1. Mô tả thư mục static	44
4.1.2. Mô tả thư mục source	44
4.1.3. Mô tả thư mục infra	47
4.2. Các yêu cầu về cài đặt	50
4.3. Triển khai hệ thống trên môi trường đám mây	50
4.4. Đánh giá hiệu quả và kiểm thử hệ thống	54
4.4.1. Hiệu quả của hàm Lambda	54
4.4.2. Kiểm thử chức năng Đăng nhập SSO	58
4.4.3. Kiểm thử chức năng phía Quản trị viên	60
4.5. Tổng kết chương	62
KẾT LUẬN	63
TÀI LIỆU THAM KHẢO	65

DANH MỤC HÌNH VẼ

Hình 1.1. Minh họa hoạt động của hệ thống SSO phi máy chủ	6
Hình 1.2. Minh họa cách Terraform vận hành [7]	13
Hình 1.3. Đoạn mã Terraform tạo máy chủ ảo trên AWS	14
Hình 2.1. Biểu đồ ca sử dụng của hệ thống	19
Hình 3.1. Kiến trúc tổng quan của hệ thống	25
Hình 3.2. Mô hình kiến trúc mức cao	26
Hình 3.3. Mô tả luồng tổng quan của hệ thống	27
Hình 3.4. Dữ liệu về nhóm phân quyền lưu trữ trên Amazon Cognito	29
Hình 3.5. Dữ liệu về người dùng lưu trữ trên Amazon Cognito	31
Hình 3.6. Dữ liệu được lưu trữ tại Amazon DynamoDB	33
Hình 3.7. Một số dữ liệu lưu trữ trên Parameter Store	33
Hình 3.8. Biểu đồ luồng Đăng nhập	34
Hình 3.9. Biểu đồ luồng Tạo tài khoản	35
Hình 3.10. Biểu đồ luồng Thiết lập thông tin đăng nhập	36
Hình 3.11. Biểu đồ luồng Quản lý tài khoản – Sửa, xoá người dùng	37
Hình 3.12. Biểu đồ luồng Quản lý tài khoản – Yêu cầu thay đổi mật khẩu	38
Hình 3.13. Biểu đồ luồng Quản lý nhóm phân quyền - Tạo, xoá nhóm	39
Hình 3.14. Biểu đồ luồng Thêm, xoá quyền thành viên nhóm của người dùng	40
Hình 3.15. Biểu đồ luồng Đăng xuất	41
Hình 4.1. Mô tả cấu trúc cây mã nguồn của hệ thống	44
Hình 4.2. Mẫu một hàm tuỳ biến được định nghĩa trong lớp Lambda	45
Hình 4.3. Mẫu mã nguồn hàm Lambda yêu cầu đặt mật khẩu	46
Hình 4.4. Mẫu cấu hình biến môi trường cho một hàm Lambda	47
Hình 4.5. Cấu trúc thư mục common	48
Hình 4.6. Một phần mã nguồn định nghĩa hàm Lambda	49
Hình 4.7. Tập định nghĩa Terragrunt cho môi trường dev	49
Hình 4.8. Khởi tạo môi trường thực thi	51
Hình 4.9. Áp dụng tệp cấu hình và khởi tạo tài nguyên	53
Hình 4.10. Cấu hình kiểm tra chịu tải	56

Hình 4.11. Cài đặt và chạy Artillery.....	56
Hình 4.12. Kết quả kiểm thử chịu tải	57
Hình 4.13. Hàm Lambda mở rộng số lượng môi trường thực thi.....	57
Hình 4.14. Giao diện Đăng nhập SSO.....	59

DANH MỤC BẢNG BIỂU

Bảng 3.1. Các thông tin về nhóm phân quyền.....	30
Bảng 3.2. Các thông tin về người dùng	32
Bảng 3.3. Mô tả thông tin lưu trữ trên DynamoDB.....	32
Bảng 4.1. So sánh mức giá Lambda và EC2	58
Bảng 4.2. Danh sách các ca kiểm thử Đăng nhập SSO	60
Bảng 4.3. Kết quả các ca kiểm thử Đăng nhập SSO	60
Bảng 4.4. Danh sách ca kiểm thử chức năng phía Quản trị viên.....	61
Bảng 4.5. Kết quả các ca kiểm thử chức năng phía Quản trị viên.....	61

DANH MỤC CỤM TỪ VIẾT TẮT

Ký hiệu	Từ gốc	Ý nghĩa
ACL	Access Control List	Danh sách kiểm soát truy cập
API	Application Programming Interface	Giao diện lập trình ứng dụng
ARN	Amazon Resource Name	
AWS	Amazon Web Services	
CDN	Content Delivery Network	Mạng phân phối nội dung
CSDL	Cơ sở dữ liệu	
DRY	Don't Repeat Yourself	
IaC	Infrastructure as Code	Hạ tầng dưới dạng mã
IP	Internet Protocol	Giao thức Internet
JWT	JSON Web Token	
KLTN	Khoá luận tốt nghiệp	
LDAP	Lightweight Directory Access Protocol	
LDIF	LDAP Data Interchange Format	
MFA	Multi-factor Authentication	Xác thực đa yếu tố
OIDC	OpenID Connect	
OSI	Open Systems Interconnect	
PC	Personal Computer	Máy tính cá nhân
QR	Quick Response	
REST	Representational State Transfer	
SAML	Security Assertion Markup Language	
SDK	Software Development Kit	Công cụ phát triển phần mềm
SSO	Single Sign-On	Đăng nhập một lần

MỞ ĐẦU

1. Đặt vấn đề

Để xác thực định danh người dùng, một hệ thống có thể sử dụng một trong ba yếu tố sau: yếu tố biết (something you know), yếu tố sở hữu (something you have) và yếu tố sinh trắc học (something you are) [1]. Trong đó, yếu tố biết (ví dụ như tài khoản/mật khẩu, câu hỏi bí mật,...) là yếu tố phổ biến được các hệ thống sử dụng để xác thực định danh người dùng. Cùng với sự bùng nổ và phát triển của hệ thống thông tin hiện nay, một người dùng có thể sẽ phải ghi nhớ rất nhiều tài khoản/mật khẩu cho mỗi ứng dụng, dịch vụ mà họ sử dụng. Điều này không những ảnh hưởng không nhỏ đến trải nghiệm người dùng mà còn đặt ra những nguy cơ bảo mật trong việc người dùng quản lý và lưu trữ những thông tin định danh nhạy cảm như vậy.

Trong xu hướng chung của các hệ thống về việc chia sẻ chung dữ liệu, việc chia sẻ chung dữ liệu định danh người dùng trở thành một xu hướng phổ biến giúp đơn giản hoá quá trình định danh người dùng cũng như tạo ra một môi trường liền mạch và thân thiện với người dùng. Từ đó nhu cầu cho các hệ thống xác thực một lần (Single Sign-On/ SSO) trở nên vô cùng cần thiết. Các hệ thống SSO cung cấp khả năng cho người dùng có thể truy cập những dịch vụ khác nhau trên hệ thống mạng chỉ với một tài khoản định danh duy nhất. Người dùng khi này chỉ cần đăng nhập một lần trên hệ thống xác thực SSO, và khi truy cập vào một hệ thống dịch vụ khác có liên kết với hệ thống định danh SSO trong cùng phiên định danh, người dùng có thể truy cập ngay vào hệ thống đó mà không cần phải trải qua các bước đăng ký định danh hay xác thực định danh.

Hiện tại có rất nhiều giải pháp SSO khác nhau được cung cấp bởi các bên thứ ba (như Google, Facebook, Twitter, OAuth,...) cung cấp các hệ thống xác thực SSO. Tuy vậy, nhiều doanh nghiệp có chính sách đặc thù, đòi hỏi các doanh nghiệp này phải tự xây dựng cơ chế SSO cho các dịch vụ nội bộ của doanh nghiệp. Vấn đề đặt ra với các doanh nghiệp như vậy đó là việc xây dựng và quản lý một hệ thống SSO truyền thống yêu cầu chi phí tương đối cao cho hoạt động phát triển cũng như vận hành hệ thống.

2. Giải pháp đề xuất

Hệ thống SSO phi máy chủ được xây dựng dựa trên hạ tầng đám mây, tận dụng các dịch vụ được quản lý (managed service) và phi máy chủ (serverless) của AWS¹. Việc đưa hệ thống lên môi trường đám mây nói chung và sử dụng các dịch vụ điện toán phi máy chủ nói riêng không những giúp tối ưu chi phí khi vận hành và phát triển hệ

¹ <https://aws.amazon.com/what-is-aws/>

thống mà còn giúp hệ thống tận dụng được những ưu điểm của nền tảng đám mây như khả năng bảo mật, khả năng giãn nở tự động,... Ngoài ra hệ thống SSO được phát triển cũng sẽ hướng tới mô hình quản lý tài nguyên đám mây bằng mã nguồn (IaC)² giúp tự động hoá quá trình phát triển hệ thống.

3. Kết quả đạt được

Khoá luận này sẽ tập trung vào phần phát triển kiến trúc và hoạt động phía mặt sau của hệ thống. Phiên bản đầu tiên của hệ thống được triển khai cho phép người dùng chỉ cần đăng nhập một lần duy nhất trên hệ thống SSO và sau đó có thể truy cập vào các hệ thống dịch vụ khác nhau của doanh nghiệp mà không cần phải đăng nhập lại. Phiên bản này cũng cung cấp khả năng Quản trị viên thêm người dùng mới vào hệ thống khi có nhân viên mới gia nhập doanh nghiệp, phân quyền những người dùng này vào các nhóm chức năng khác nhau.

4. Cấu trúc khoá luận

Sau phần *Mở đầu* giới thiệu về vấn đề nghiên cứu, khoá luận sẽ được trình bày với 04 chương nội dung:

Chương 1: Cơ sở lý thuyết – Giới thiệu về Single Sign-On cũng như các dịch vụ và nền tảng công nghệ sẽ sử dụng để xây dựng hệ thống.

Chương 2: Phân tích yêu cầu – Mô tả bài toán, phân tích các yêu cầu của bài toán, từ đó đưa ra các ca sử dụng.

Chương 3: Thiết kế hệ thống – Trình bày thiết kế luồng cũng như mô hình kiến trúc của hệ thống dựa trên những dịch vụ và nền tảng đã trình bày ở Chương 1.

Chương 4: Triển khai hệ thống, cài đặt và kiểm thử – Tập trung vào việc trình bày mô hình mã nguồn để triển khai hệ thống, cài đặt và đưa hệ thống lên môi trường đám mây cũng như đưa ra các kết quả đã đạt được.

Phần *Kết luận* sẽ tổng kết lại những vấn đề được trình bày trong 04 chương nội dung kể trên, những kết quả đạt được của khoá luận cũng như hướng nghiên cứu tiếp theo để phát triển hệ thống.

² <https://www.redhat.com/en/topics/automation/what-is-infrastructure-as-code-iac>

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT

Trong chương này, KLTN sẽ giới thiệu về cơ sở lý thuyết và những nền tảng công nghệ để xây dựng hệ thống SSO phi máy chủ, tập trung vào ba khía cạnh: Thứ nhất, giới thiệu về Single Sign-On, các mô hình triển khai Single Sign-On hiện tại cũng như mô hình triển khai Single Sign-On sẽ được triển khai trong KLTN. Thứ hai, giới thiệu về mô hình phi máy chủ cũng như một số dịch vụ đám mây trên AWS hỗ trợ phát triển hệ thống. Thứ ba, giới thiệu về các công cụ hỗ trợ cấp phát và quản lý tài nguyên đám mây bằng mã nguồn (IaC), cụ thể là Terraform và đánh giá hiệu quả của nó trong việc đơn giản hoá và tự động hoá quá trình cấp phát tài nguyên đám mây đáp ứng những yêu cầu mà bài toán đặt ra.

1.1. Single Sign-On

1.1.1. Giới thiệu chung về Single Sign-On

SSO là một cơ chế xác thực mà trong đó người dùng đăng nhập vào một hệ thống và được trao quyền truy cập một cách tự động vào các hệ thống khác [2]. Trước khi có sự xuất hiện của SSO, một người dùng sử dụng nhiều sản phẩm phần mềm thường phải có một bộ tài khoản/mật khẩu cho mỗi ứng dụng (trong nhiều trường hợp, người dùng sẽ dùng chung một bộ tài khoản/mật khẩu cho tất cả các ứng dụng mà mình sử dụng). Khi muốn truy cập vào mỗi ứng dụng, người dùng sẽ phải nhập tài khoản/mật khẩu để đăng nhập vào ứng dụng. Điều này thực sự là một công việc tốn thời gian, nhất là trong môi trường doanh nghiệp khi mỗi nhân viên sẽ phải nhập lại tài khoản/mật khẩu mỗi khi muốn đăng nhập vào các ứng dụng, phần mềm khác nhau.

Hơn hết, việc này cũng làm gia tăng các vấn đề về bảo mật. Trong trường hợp người dùng sử dụng chung một bộ tài khoản/mật khẩu cho tất cả các ứng dụng, khi người dùng đăng ký tài khoản trên một ứng dụng không an toàn và sử dụng chung một bộ tài khoản/mật khẩu với các ứng dụng khác, người dùng sẽ có nguy cơ trong việc đánh mất thông tin định danh và cấp quyền truy cập vào các ứng dụng khác cho kẻ đánh cắp. Với trường hợp sử dụng các bộ tài khoản/mật khẩu khác nhau cho các ứng dụng đòi hỏi người dùng cần phải ghi nhớ khối lượng lớn thông tin về định danh. Khi này người dùng có xu hướng lưu các thông tin định danh này vào một nơi lưu trữ chung (ví dụ như sổ tay, ứng dụng ghi chép của điện thoại,...). Điều này cũng sẽ tạo ra những lỗ hổng bảo mật nếu những thông tin định danh này không được lưu trữ một cách an toàn, gây ra mất cắp thông tin.

Do vậy, với các hệ thống bao gồm nhiều ứng dụng phần mềm khác nhau (nhất là trong các doanh nghiệp), việc có chung một bộ tài khoản định danh và việc yêu cầu

người dùng chỉ cần đăng nhập một lần giúp giảm thiểu rất nhiều thời gian trong việc xác thực người dùng cũng như đem lại những lợi ích nhất định về mặt an ninh và bảo mật.

Tựu chung lại, SSO có thể đem lại một số lợi ích sau:

- Giảm thời gian người dùng cần bỏ ra cho hoạt động đăng nhập vào các dịch vụ khác nhau được cung cấp trong các hệ thống phân tán.
- Tăng cường tính bảo mật bằng việc người dùng không phải nhớ nhiều thông tin đăng nhập cho các dịch vụ, ứng dụng khác nhau.
- Việc quản lý người dùng đơn giản hơn cho các Quản trị viên, tránh trùng lặp thông tin của cùng một người dùng trên các ứng dụng khác nhau.
- Tạo nên sự đồng bộ cho các ứng dụng và dịch vụ trong cùng một hệ thống, làm mượt mà hơn trải nghiệm của người dùng.

Mặc dù những lợi ích kể trên, SSO cũng có thể là một con dao hai lưỡi với những nhược điểm như:

- Nếu hệ thống SSO tồn tại những lỗ hổng về bảo mật và bị tấn công, tất cả những hệ thống sử dụng nó cũng sẽ phải chịu rủi ro về an ninh và bảo mật, có thể cho phép tin tặc đồng thời truy cập được vào tất cả ứng dụng và dịch vụ trong hệ thống.
- Chi phí để triển khai các hệ thống SSO thường khá tốn kém, cả về phần cứng, phần mềm và cả về nguồn nhân lực trong quá trình phát triển và vận hành.

1.1.2. Một số giải pháp Single Sign-On

Để có thể triển khai một hệ thống SSO, sẽ có một vài những tiêu chuẩn và giao thức mà một hệ thống cần phải tuân thủ, trong đó ta có thể kể tới một vài giao thức tiêu chuẩn trong SSO như [2]:

- **Security Assertion Markup Language (SAML):** Đây là một trong những giao thức phổ biến nhất trong các mô hình triển khai SSO. Giao thức này hỗ trợ trao đổi các thông tin xác thực và phân quyền theo định dạng XML. Mô hình này được triển khai với ba thành phần: người dùng (user), nhà cung cấp định danh (identity provider) và nhà cung cấp dịch vụ (service provider). Khi người dùng gửi các yêu cầu tới các nhà cung cấp dịch vụ, nhà cung cấp dịch vụ sẽ kiểm tra quyền truy cập của người dùng với nhà cung cấp định danh trước khi cấp quyền truy cập cho người dùng vào tài nguyên và thực hiện các yêu cầu của người dùng. Mô hình này đảm bảo độ tin cậy bằng sự xác thực (assertion) giữa nhà cung cấp dịch vụ và nhà cung cấp định danh.

- **OpenID Connect (OIDC)**: Đây là giao thức xác thực được sử dụng khá phổ biến trong khoảng thời gian trở lại đây, hoạt động dựa trên giao thức tiêu chuẩn OAuth 2.0³. Giao thức này xử lý các tác vụ xác thực thông qua JSON Web Tokens (JWTs)⁴. Tương đồng với SAML, OIDC cung cấp quyền truy cập cho người dùng thông qua các nhà cung cấp định danh tập trung với mô hình tương tự mô hình của SAML. Tuy vậy luồng xác thực của OIDC cũng có những điểm khác biệt so với SAML. Khi người dùng gửi yêu cầu đến một ứng dụng, ứng dụng này sẽ gửi một yêu cầu xác thực đến nhà cung cấp định danh. Nhà cung cấp định danh xác thực thông tin của người dùng và cấp phát cho người dùng một mã xác thực gọi là “id_token”. Mã xác thực này chứa thông tin định danh của người dùng. Các ứng dụng nhận được mã xác thực sẽ sử dụng mã xác thực này để cấp quyền truy cập cho người dùng. Xác thực dựa trên JWT giúp mã xác thực trở nên gọn nhẹ, dễ đọc hơn so với việc sử dụng các thư viện khá nặng để phân tích cú pháp từ mã xác thực dạng XML của SAML vì vậy hiện nay giao thức OIDC hoạt động dựa trên JWT có được sự phổ biến và ưa chuộng hơn so với việc sử dụng giao thức SAML.

- **Lightweight Directory Access Protocol (LDAP)**: Đây là một giao thức ứng dụng lâu đời nhất được sử dụng để truy cập và duy trì các dịch vụ thông tin thư mục thông qua mạng IP, thường thấy trong các doanh nghiệp sử dụng mạng nội bộ (intranet). LDAP cho phép việc truy cập vào các thư mục tập trung chứa thông tin định danh được chia sẻ giữa các ứng dụng khác nhau. LDAP thường được sử dụng kết hợp với Active Directory (AD)⁵. LDAP hoạt động theo mô hình client-server, các ứng dụng khách gửi yêu cầu (ví dụ như xác thực, cập nhật thông tin người dùng,...) đến máy chủ LDAP; máy chủ LDAP sẽ nhận yêu cầu và thực hiện tìm kiếm và trả lại kết quả cho ứng dụng khách. Các thông tin được trao đổi trong giao thức LDAP thường dưới dạng LDAP Data Interchange Format (LDIF).

1.2. Giải pháp Single Sign-On phi máy chủ

1.2.1. Giới thiệu tổng quan

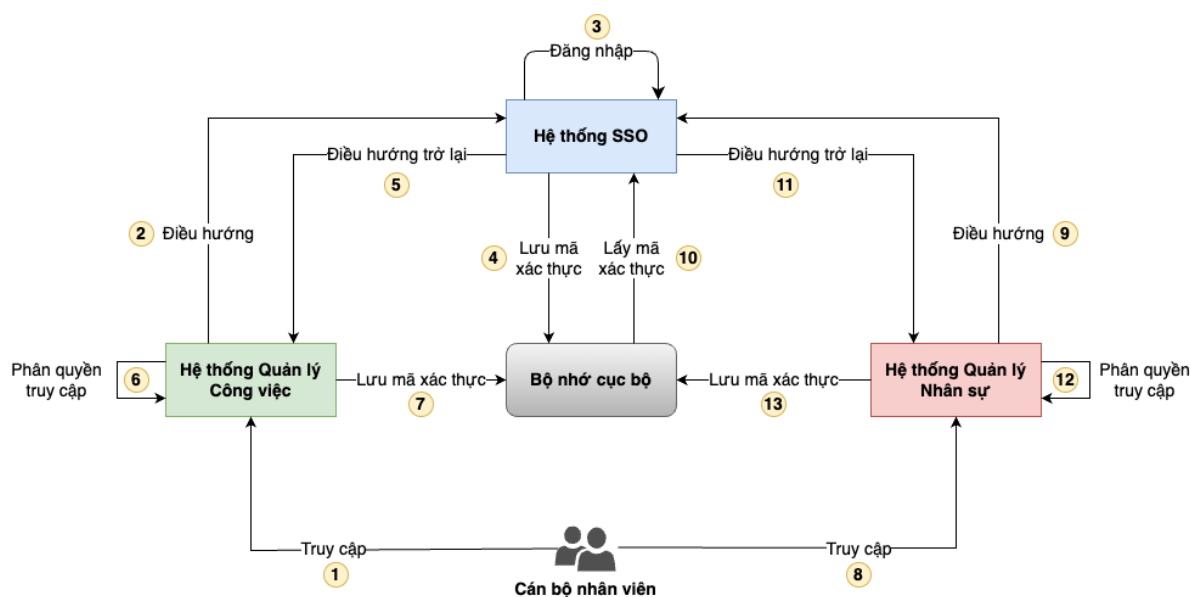
Mô hình triển khai Single Sign-On phi máy chủ vẫn đảm bảo yêu cầu cơ bản của một hệ thống SSO – người dùng chỉ cần đăng nhập một lần duy nhất và có thể truy cập vào các hệ thống, dịch vụ được cho phép mà không cần đăng nhập lại. Để hiểu rõ hơn

³ <https://auth0.com/intro-to-iam/what-is-oauth-2>

⁴ <https://jwt.io/introduction>

⁵ <https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/get-started/virtual-dc/active-directory-domain-services-overview>

phương thức hoạt động của hệ thống sẽ được phát triển trong KLTN, ta có thể tham khảo ví dụ minh họa ở Hình 1.1 bên dưới.



Hình 1.1. Minh họa hoạt động của hệ thống SSO phi máy chủ

1. Người dùng truy cập vào Hệ thống Quản lý công việc.
2. Hệ thống này điều hướng người dùng tới hệ thống đăng nhập SSO phi máy chủ.
3. Tại hệ thống SSO phi máy chủ, người dùng tiến hành nhập thông tin đăng nhập để xác thực.
4. Hệ thống SSO xác thực người dùng và lưu mã xác thực của người dùng vào bộ nhớ cục bộ (Local Storage).
5. Hệ thống SSO điều hướng người dùng trở lại hệ thống Quản lý công việc cùng với mã xác thực người dùng.
6. Hệ thống Quản lý công việc phân quyền truy cập cho người dùng vào hệ thống dựa trên các nhóm phân quyền được chỉ định theo mã xác thực người dùng được cấp bởi hệ thống SSO.
7. Hệ thống Quản lý công việc lưu lại mã xác thực vào bộ nhớ cục bộ cho những lần truy cập tiếp theo.
8. Cán bộ nhân viên truy cập vào Hệ thống Quản lý nhân sự.
9. Hệ thống điều hướng truy cập tiếp hệ thống SSO phi máy chủ.
10. Hệ thống SSO kiểm tra mã xác thực lưu trong bộ nhớ cục bộ và xác nhận người dùng đã đăng nhập.
11. Hệ thống SSO trả lại mã xác thực cho Hệ thống Quản lý nhân sự.

12. Hệ thống Quản lý nhân sự phân quyền truy cập cho người dùng vào hệ thống dựa trên các nhóm phân quyền được chỉ định theo mã xác thực người dùng được cấp bởi hệ thống SSO.

13. Hệ thống Quản lý nhân sự lưu lại mã xác thực vào bộ nhớ cục bộ cho những lần truy cập tiếp theo.

Như vậy, theo mô hình minh họa đã được trình bày, người dùng chỉ cần đăng nhập một lần và có thể sử dụng mã xác thực được cấp ở lần đăng nhập trước đó để truy cập vào các dịch vụ, hệ thống nội bộ khác của doanh nghiệp.

1.2.2. Điểm mạnh của hệ thống SSO phi máy chủ

Hệ thống được xây dựng dựa trên mô hình phát triển sử dụng các dịch vụ điện toán phi máy chủ. Ta sẽ làm rõ hơn về điện toán phi máy chủ cũng như các dịch vụ đám mây sẽ sử dụng ở trong các phần tiếp theo của chương này. Việc xây dựng hệ thống SSO dưới dạng phi máy chủ có những điểm mạnh sau:

- Tận dụng được sức mạnh và tính linh hoạt của các hệ thống điện toán đám mây⁶.
- Tiết kiệm chi phí vận hành, máy chủ hệ thống chỉ chạy khi có yêu cầu nhưng vẫn đáp ứng được khả năng phản hồi nhanh chóng với độ trễ không đáng kể.
- Có thể mở rộng nhanh chóng, có thể đáp ứng được tới hàng nghìn yêu cầu trong một khoảng thời gian tận dụng khả năng vận hành đồng bộ của dịch vụ AWS Lambda⁷ mà ta sẽ giới thiệu chi tiết hơn ở phần sau.
- Rút ngắn thời gian phát triển phần mềm với các dịch vụ được quản lý bởi AWS, giảm thiểu thời gian và chi phí bỏ ra cho hoạt động vận hành.
- Khả năng phục hồi sau sự cố tốt, hệ thống vẫn có thể vận hành khi có máy chủ phía AWS xảy ra sự cố mà không gây gián đoạn.
- Có thể kết hợp với nhiều dịch vụ xác thực khác như Google, Facebook, Microsoft Active Directory,...
- Phù hợp với đặc điểm bài toán xác thực người dùng sẽ được làm rõ ở Chương 4.

1.3. Điện toán phi máy chủ

Điện toán phi máy chủ (serverless computing) là mô hình thực thi và phát triển ứng dụng điện toán đám mây cho phép các nhà phát triển xây dựng và chạy mã nguồn của ứng dụng mà không cần cấp phát và quản lý máy chủ cũng như cơ sở hạ tầng mặt

⁶ <https://docs.aws.amazon.com/whitepapers/latest/aws-overview/six-advantages-of-cloud-computing.html>

⁷ <https://docs.aws.amazon.com/lambda/latest/dg/configuration-concurrency.html>

sau (backend) [3]. Nói cách khác, trong kiến trúc phi máy chủ (thực chất vẫn cần sử dụng máy chủ), thay vì phải trả chi phí cho việc sử dụng bằng thông hay thuê máy chủ cố định, các hệ thống phi máy chủ cho phép loại bỏ các nhiệm vụ quản lý cơ sở hạ tầng và chỉ tính phí sử dụng dựa trên mức sử dụng. Công nghệ phi máy chủ cũng cung cấp tính năng tự động mở rộng quy mô, cung cấp tính sẵn sàng cao được tích hợp sẵn và mô hình thanh toán theo nhu cầu để sử dụng nhằm tăng tính nhanh nhạy và tối ưu hóa chi phí [4].

1.4. Amazon Cognito

Amazon Cognito⁸ là một dịch vụ được quản lý hoàn toàn thuộc nhóm *Bảo mật, định danh và tuân thủ* của AWS, cung cấp tính năng xác thực, phân quyền và quản lý người dùng cho các ứng dụng web và di động. Cognito lưu trữ và quản lý thông tin định danh người dùng trong các thư mục người dùng gọi là nhóm người dùng (User Pool)/nhóm định danh (Identity Pool). Amazon Cognito cung cấp cơ chế quản lý và xác thực người dùng bằng tên người dùng/email và mật khẩu được lưu trữ trong User Pool hoặc thông qua bên thứ ba, chẳng hạn như Facebook, Amazon hoặc Google cũng như các nhà cung cấp định danh khác thông qua SAML và OIDC.

Để đơn giản hóa bài toán, hệ thống SSO phi máy chủ sẽ chỉ sử dụng chức năng User Pool của Cognito để lưu trữ, quản lý, và phân quyền người dùng trong hệ thống nội bộ của doanh nghiệp. Quản trị viên có thể truy cập vào giao diện của Amazon Cognito để thêm một/một số nhóm người dùng mới, phân quyền những người dùng này vào các nhóm chức năng khác nhau, cũng như quản lý các thông tin cá nhân của người dùng. Ngoài ra, để phát triển bài toán đáp ứng yêu cầu mức cao hơn của các doanh nghiệp thực tế, ta có thể tích hợp các thư mục người dùng có trong Active Directory lưu trữ trong các máy chủ tại chỗ (on-premises) của doanh nghiệp.

1.5. AWS Lambda

AWS Lambda⁹ là một dịch vụ điện toán phi máy chủ, hướng sự kiện, cho phép chạy mã nguồn cho hầu hết mọi loại ứng dụng hoặc dịch vụ mặt sau mà không cần khởi tạo hay quản lý máy chủ. Khi có yêu cầu thực thi, AWS sẽ khởi tạo môi trường thực thi và tiến hành thực thi mã nguồn đã được cung cấp, trong khi đó trong thời gian chờ, AWS sẽ tiến hành dừng môi trường thực thi. Vì vậy, về mặt kinh tế, việc sử dụng AWS Lambda giúp người dùng chỉ phải trả phí sử dụng cho thời gian thực thi thực tế của

⁸ <https://aws.amazon.com/cognito/>

⁹ <https://aws.amazon.com/lambda/>

Lambda (tính theo mili giây) cũng như tối ưu hóa hiệu suất và thời gian thực thi với kích thước bộ nhớ phù hợp với nhu cầu cá nhân. AWS chịu trách nhiệm quản lý toàn bộ cơ sở hạ tầng, cung cấp sự cân bằng về bộ nhớ, CPU, kết nối mạng, và những tài nguyên khác đảm bảo chạy mã trên cơ sở hạ tầng có độ sẵn sàng và khả năng chịu lỗi cao, vì vậy lập trình viên sẽ không phải bận tâm tới các vấn đề liên quan tới phần cứng mà chỉ cần tập trung vào hoạt động phát triển mã nguồn. AWS Lambda cũng là một ví dụ về dịch vụ hỗ trợ container của AWS. AWS Lambda cung cấp khả năng vận hành cho nhiều trường hợp sử dụng đa dạng như xử lý file, xử lý luồng, ứng dụng web,... nằm trong giới hạn về tài nguyên và thời gian thực thi của Lambda (tối đa 15 phút)¹⁰.

Đơn vị cơ bản của Lambda chính là các hàm Lambda (Lambda function). Hàm Lambda là nơi tiếp nhận và xử lý các sự kiện được gửi đến Lambda. Khi một sự kiện (ví dụ như yêu cầu API, một công việc định kỳ được lập lịch,...) kích hoạt một hàm Lambda, AWS Lambda sẽ khởi tạo một môi trường thực thi tách biệt quản lý các tài nguyên như bộ nhớ, CPU,... cũng như thực thi mã nguồn của hàm Lambda. Môi trường thực thi của AWS Lambda cho phép lập trình với nhiều ngôn ngữ lập trình phổ biến như: Python, Nodejs, Java, Go, C#, Ruby, PowerShell cũng như cho phép tùy biến cài đặt và sử dụng bất cứ một ngôn ngữ lập trình nào khác để khởi tạo các hàm. Lambda cũng cho phép đóng gói mã (khung phần mềm, bộ phát triển phần mềm SDK, thư viện,...) dưới dạng lớp Lambda (Lambda layer), giúp lập trình viên dễ dàng quản lý mã nguồn cũng như tăng khả năng tái sử dụng mã nguồn trên nhiều hàm Lambda khác nhau.

Hệ thống trong bài toán này sẽ sử dụng Python làm ngôn ngữ lập trình cho các hàm Lambda cũng như các lớp Lambda. Python hỗ trợ rất nhiều thư viện cho lập trình viên cũng như có một cộng đồng hỗ trợ hoạt động tích cực. Hơn hết, Python giúp làm giảm thời gian khởi động môi trường thực thi của Lambda (gọi là thời gian “cold start”¹¹) của hàm Lambda, từ đó làm giảm thời gian thực thi, tăng tốc độ xử lý và trả kết quả cho người dùng¹².

1.6. Amazon API Gateway

Amazon API Gateway¹³ là dịch vụ được quản lý hoàn toàn giúp các nhà phát triển dễ dàng tạo, triển khai, duy trì, giám sát và bảo vệ API ở mọi quy mô. Bằng cách sử dụng API Gateway, ta có thể tạo các REST API và WebSocket API để kích hoạt các

¹⁰ <https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html>

¹¹ <https://docs.aws.amazon.com/lambda/latest/operatorguide/execution-environments.html#cold-start-latency>

¹² <https://mikhail.io/serverless/coldstarts/aws/languages/>

¹³ <https://aws.amazon.com/api-gateway/>

ứng dụng giao tiếp hai chiều theo thời gian thực. API Gateway hỗ trợ các tác vụ xử lý có trong container và các dịch vụ phi máy chủ (như AWS Lambda). API Gateway xử lý tất cả các tác vụ liên quan đến tiếp nhận và xử lý lên đến hàng trăm nghìn lệnh gọi API đồng thời, bao gồm quản lý lưu lượng truy cập, hỗ trợ CORS, xác thực và kiểm soát truy cập, điều tiết, giám sát và quản lý phiên bản API. Trong bài toán này, ta sử dụng API Gateway để nhận các yêu cầu từ phía người dùng mặt trước (frontend) của ứng dụng web chuyển nó cho AWS Lambda xử lý.

1.7. Amazon S3

Amazon Simple Storage Service (Amazon S3)¹⁴ là một dịch vụ lưu trữ đối tượng (object storage). S3 cho phép lưu trữ lượng dữ liệu với kích thước mỗi đối tượng lên tới 5TB và hầu như không giới hạn về số lượng đối tượng lưu trữ, đảm bảo độ bền dữ liệu đạt mười một chữ số 9 (99,999999999%) cũng như độ khả dụng trong một năm lên tới 99,99% thông qua hơn 4 tỷ phép tính toán checksum mỗi giây¹⁵. Các tệp dữ liệu được lưu trữ dưới dạng các đối tượng và được lưu trong các thùng dữ liệu (gọi là bucket). S3 có thể được sử dụng để lưu trữ và phân phối nội dung web và phương tiện, lưu trữ các trang web tĩnh, lưu trữ dữ liệu để tính toán và phân tích, sao lưu và lưu trữ dữ liệu quan trọng. S3 cũng cung cấp nhiều cơ chế lưu trữ dữ liệu phù hợp với nhiều mục đích sử dụng khác nhau như S3 Standard, S3 Standard-Infrequent Access, Amazon S3 Glacier, Amazon S3 Glacier Deep Archive,... Hệ thống của bài toán sẽ sử dụng Amazon S3 để lưu trữ các trang web đăng nhập mặt trước của hệ thống cũng như phân phối nội dung (ảnh, video,...) cho ứng dụng mặt trước.

1.8. Amazon CloudFront

Amazon CloudFront¹⁶ là một dịch vụ CDN của AWS giúp tăng tốc độ phân phối nội dung của các trang web (tĩnh và động), chẳng hạn như các tệp .html, .css, .js cũng như các nội dung đa phương tiện tới người dùng cuối. CloudFront phân phối nội dung qua mạng lưới hơn 400 cụm trung tâm dữ liệu của AWS trải khắp trên toàn cầu được gọi là các vị trí biên (edge location). Khi người dùng gửi yêu cầu truy vấn dữ liệu được phân phối bằng CloudFront, yêu cầu sẽ được chuyển đến vị trí biên cung cấp độ trễ thấp nhất để nội dung được phân phối với hiệu suất tốt nhất có thể nếu dữ liệu đã được lưu vào bộ nhớ đệm (cache) trong các trung tâm dữ liệu của AWS.

¹⁴ <https://aws.amazon.com/s3/>

¹⁵ <https://aws.amazon.com/blogs/aws/celebrate-amazon-s3s-17th-birthday-at-aws-pi-day-2023/>

¹⁶ <https://aws.amazon.com/cloudfront/>

Với đặc điểm các dịch vụ và hệ thống nội bộ của doanh nghiệp sẽ cần điều hướng cũng như truy vấn đến hệ thống SSO với tần suất thường xuyên để xác thực định danh cũng kiểm tra mã xác thực của người dùng, KLTN sẽ sử dụng CloudFront làm dịch vụ phân phối nội dung cho các trang web đăng nhập mặt trước của hệ thống SSO được lưu trữ trong S3 cũng như các API cho mặt sau thông qua API Gateway. Việc sử dụng CloudFront (với hai vị trí biên phân phối nội dung được đặt tại Hà Nội và Thành phố Hồ Chí Minh) sẽ giúp làm giảm độ trễ một cách hiệu quả khi người dùng truy cập vào hệ thống, thay vì phải truy vấn trực tiếp đến các trung tâm dữ liệu vận hành các dịch vụ của AWS được đặt tại các khu vực (Region) của AWS.

1.9. AWS WAF

Bên cạnh Amazon Cognito, AWS WAF¹⁷ cũng là một trong số các dịch vụ thuộc nhóm *Bảo mật, định danh và tuân thủ* của AWS. Như cái tên Web Application Firewall (WAF) mô tả, AWS WAF là dịch vụ bảo mật hoạt động ở tầng 3, 4 và 7 của mô hình OSI, hỗ trợ bảo vệ các trang web khỏi các dạng tấn công phổ biến như: cross-site scripting, SQL injection,... AWS WAF cung cấp khả năng giám sát các truy vấn HTTP(S) được chuyển tiếp tới các tài nguyên ứng dụng web thông qua các nguồn như Amazon CloudFront hay Amazon API Gateway,... WAF cho phép người dùng kiểm soát truy cập vào các tài nguyên được bảo mật thông qua danh sách kiểm soát truy cập (ACL). Trong thời đại an ninh thông tin đóng vai trò hàng đầu trong hoạt động thiết kế và xây dựng hệ thống công nghệ thông tin, việc sử dụng AWS WAF mang lại những lợi ích đáng kể. Ta có thể sử dụng WAF để thiết lập các quy định kiểm soát truy cập như: địa chỉ IP nguồn, quốc gia truy cập, số lượng truy cập trong một khoảng thời gian hay thậm chí sử dụng các biểu thức chính quy để lọc ra các yêu cầu có nội dung không phù hợp. Chỉ cần cấu hình đơn giản, tất cả các thao tác kiểm soát truy cập đều được thực hiện hoàn toàn bởi phía AWS vì vậy WAF được sử dụng trong hệ thống này giúp hỗ trợ gia tăng bảo mật cho phía mặt sau của ứng dụng cũng như tránh được các hành động tấn công làm ảnh hưởng tới dữ liệu và tính sẵn sàng của hệ thống.

1.10. Amazon DynamoDB

Amazon DynamoDB¹⁸ là một dịch vụ CSDL phi quan hệ NoSQL được quản lý hoàn toàn. DynamoDB cung cấp hiệu suất nhanh chóng với khả năng mở rộng tự động và liền mạch. DynamoDB cũng có thể hoạt động trên quy mô lớn, phi máy chủ và có độ sẵn sàng cao. DynamoDB thích hợp cho các ứng dụng xử lý khối lượng lớn dữ liệu và

¹⁷ <https://aws.amazon.com/waf/>

¹⁸ <https://aws.amazon.com/dynamodb/>

phải mở rộng quy mô nhanh chóng. DynamoDB cũng cung cấp thông lượng cao và độ trễ thấp với khả năng truy vấn dữ liệu với tốc độ một chữ số ở cấp độ mili giây. Theo mặc định, Amazon DynamoDB sao chép dữ liệu của trên nhiều vùng khả dụng (Availability Zone) trong một khu vực của AWS duy nhất. DynamoDB cũng cung cấp khả năng tự động hết hạn dữ liệu theo các mốc thời gian (timestamp) tùy biến được lưu trữ trong từng bản ghi. Với tốc độ truy vấn nhanh chóng, hệ thống sử dụng DynamoDB để lưu trữ các thông tin về phiên đăng nhập của người dùng.

1.11. Hạ tầng dưới dạng mã IaC

1.11.1. Tổng quan về IaC

Hạ tầng dưới dạng mã (Infrastructure as Code – IaC) là quá trình cấp phát và quản lý hạ tầng công nghệ thông tin thông qua các tệp định nghĩa mà máy có thể đọc được [5]. Khi sử dụng IaC, Quản trị viên vận hành hệ thống chỉ cần viết một đoạn mã định nghĩa, triển khai, cập nhật hoặc thậm chí xóa bỏ một phần hoặc toàn bộ hạ tầng công nghệ thông tin (ví dụ như máy chủ, CSDL,...), các yêu cầu được thực hiện một cách tự động mà không cần thực hiện các thao tác thủ công.

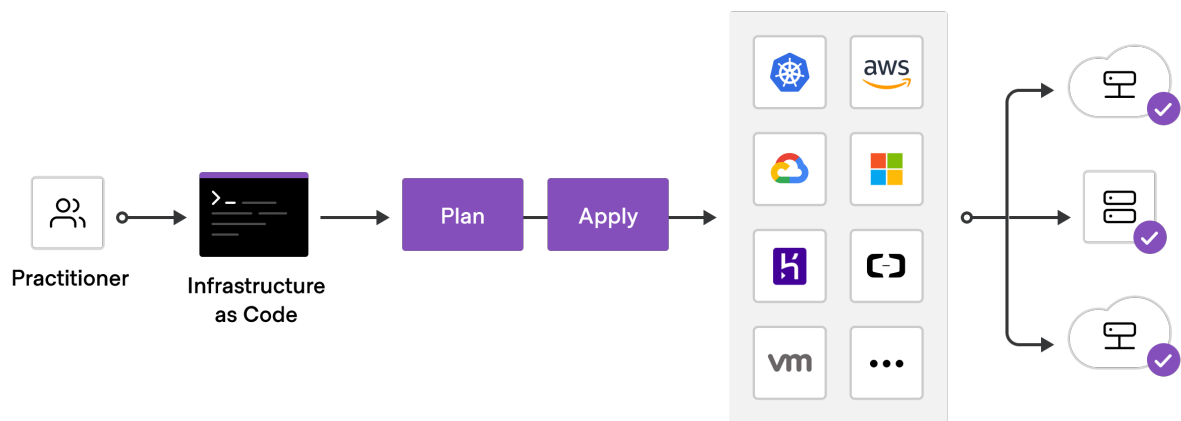
Việc sử dụng các công cụ IaC có thể mang lại nhiều lợi ích to lớn trong nỗ lực quản lý và vận hành hạ tầng cho doanh nghiệp, đặc biệt là hạ tầng đám mây. Thật vậy, IaC cung cấp khả năng tận dụng tối đa hoá lợi ích của các công cụ tự động hoá. Các quy trình tự động hoá giải phóng con người (bao gồm các Quản trị viên vận hành hệ thống) khỏi các quy trình thủ công lặp đi lặp lại gây nhàm chán. Hơn nữa, các quy trình vận hành hệ thống cũng trở nên nhất quán và tránh xảy ra những sai sót không đáng có do yếu tố con người. IaC cũng giúp gia tốc thời gian đưa các sản phẩm công nghệ thông tin của doanh nghiệp tới thị trường cũng như gia tăng khả năng đổi mới và sáng tạo của doanh nghiệp. Việc sử dụng IaC giúp doanh nghiệp dễ dàng tạo ra các môi trường thử nghiệm hạ tầng công nghệ thông tin cũng như khả năng nhanh chóng cập nhật (thêm, sửa, xóa) hạ tầng công nghệ thông tin chỉ bằng một vài câu lệnh.

Cũng bởi IaC được viết dưới dạng các đoạn mã, IaC cũng có thể coi là một nguồn tài liệu chi tiết về kiến trúc và cấu hình hạ tầng công nghệ thông tin. Điều này giúp các thành viên trong một đội nhóm quản lý hệ thống dễ dàng làm việc cùng nhau hơn với các hệ thống quản lý phiên bản (ví dụ như Git). Điều này cũng giúp hoạt động quản lý hạ tầng công nghệ thông tin của doanh nghiệp không chịu ảnh hưởng bởi sự rời đi hay gia nhập mới của một thành viên trong đội nhóm quản lý hệ thống. Các đoạn mã nguồn của IaC cũng có thể được đóng gói và tái sử dụng, tránh việc phải lặp lại các quy trình giống nhau từ đầu.

Với những lợi ích mà IaC mang lại, kết hợp với sự bùng nổ của các dịch vụ điện toán đám mây, IaC đang trở thành một công nghệ thu hút rất nhiều nhà phát triển cung cấp các công cụ giúp Quản trị viên và lập trình viên đơn giản hoá quá trình phát triển các đoạn mã quản lý hạ tầng công nghệ thông tin, có thể kể tới một số công cụ như: Ansible, Chef, Puppet, Terraform, AWS Cloud Formation,... Trong đó, Terraform nổi lên như một công cụ tiện lợi và hiệu quả với những đặc điểm nổi trội so với các công cụ kể trên.

1.11.2. Terraform

Terraform là công cụ IaC phát triển bởi HashiCorp cho phép bạn xây dựng, thay đổi và quản lý phiên bản tài nguyên đám mây và tại chỗ một cách an toàn và hiệu quả [6]. Terraform tạo và quản lý tài nguyên trên nền tảng đám mây và tại chỗ thông qua các giao diện lập trình ứng dụng (API) cũng như hỗ trợ đa dạng tài nguyên và dịch vụ cho các nhà cung cấp dịch vụ có sẵn như AWS, Azure, Google Cloud Platform, OpenStack,... Để có thể cấp phát các tài nguyên (như máy chủ, CSDL, hệ thống mạng,...) trên các nhà cung cấp dịch vụ như AWS, các Quản trị viên chỉ cần định nghĩa các tệp cấu hình (.tf) và chạy các câu lệnh của Terraform như: `terraform plan`, `terraform apply` để triển khai hạ tầng của hệ thống. Terraform sẽ thực hiện nhiệm vụ dịch các đoạn mã này thành các câu lệnh gọi API tương ứng tới các nhà cung cấp dịch vụ và thực hiện quá trình cấp phát tài nguyên (Hình 1.2).



Hình 1.2. Minh họa cách Terraform vận hành [7]

Hình 1.3 minh họa một đoạn mã Terraform sử dụng để tạo máy chủ ảo trên AWS. Với việc sử dụng Terraform, ta có thể dễ dàng tùy chỉnh cấu hình của các tài nguyên trên AWS chỉ với việc thay đổi một vài dòng mã nguồn, ví dụ như ta có thể thay đổi loại máy chủ ảo (`instance_type`) từ `t3.micro` thành `t3.medium` để đáp ứng yêu cầu về xử lý nghiệp vụ một cách nhanh chóng và thuận tiện, ngoài ra còn có thể lưu vết lại các thay

đổi để đối soát khi cần thiết (bằng việc quản lý mã nguồn sử dụng các công cụ quản lý phiên bản như Git).

main.tf	
1	resource "aws_instance" "web_server" {
2	ami = "ami-0dcc1e21636832c5d"
3	instance_type = "t3.micro"
4	}

Hình 1.3. Đoạn mã Terraform tạo máy chủ ảo trên AWS

So với các công cụ IaC khác, Terraform có một số những đặc điểm nổi bật sau [5]:

- Terraform là công cụ cấp phát tài nguyên thay vì công cụ quản lý cấu hình.
- Terraform là công cụ mã nguồn mở, cung cấp khả năng sử dụng miễn phí cho cả mục đích cá nhân và trong môi trường doanh nghiệp cũng như khả năng tùy biến cao.
- Tập cấu hình của Terraform được viết dưới dạng ngôn ngữ khai báo thay vì ngôn ngữ thủ tục.
- Terraform là công cụ độc lập về đám mây (cloud agnostic), cho phép chạy liên mạch trên bất kỳ nền tảng đám mây nào bằng cách sử dụng cùng một bộ công cụ và quy trình.

Với những đặc điểm như vậy của IaC nói chung và Terraform nói riêng, KLTN sẽ sử dụng Terraform làm công cụ hỗ trợ cấp phát các tài nguyên đám mây trên AWS cho các dịch vụ đám mây đã trình bày phía trên.

1.11.3. Terragrunt

Terragrunt¹⁹ là một lớp bổ sung cho Terraform, cung cấp các công cụ để giữ cho cấu hình của IaC tuân thủ theo nguyên tắc DRY, hoạt động với nhiều mô-đun Terraform và quản lý trạng thái từ xa. Các tệp cấu hình của Terragrunt (.hcl) tuân thủ theo cú pháp HashiCorp Configuration Language (HCL). Các tệp cấu hình của Terragrunt được cấu trúc dưới dạng các mô-đun cho phép tái sử dụng các mô-đun này trong toàn bộ hệ thống mà không cần phải định nghĩa lại nhiều lần, từ đó toàn bộ hoạt động của Terragrunt chỉ gói gọn trong một tệp cấu hình duy nhất `terragrunt.hcl` và trỏ đến các mô-đun đã được định nghĩa. Với bài toán được trình bày trong KLTN, ta sẽ sử dụng Terragrunt nhằm mô-đun hoá các thành phần hạ tầng của hệ thống, giúp tăng khả năng tái sử dụng cũng như tăng hiệu quả và tránh phải lặp lại các tệp cấu hình khi cấp phát tài nguyên cho các môi trường phát triển khác nhau (ví dụ như `dev`, `uat`, `prod`).

¹⁹ <https://terragrunt.gruntwork.io/docs/getting-started/quick-start/>

1.12. ReactJS

ReactJS²⁰ là một thư viện JavaScript mã nguồn mở được phát triển bởi Meta (hay trước đó được biết đến là Facebook), hỗ trợ xây dựng giao diện người dùng. ReactJS hỗ trợ xây dựng mặt trước của ứng dụng dưới dạng các thành phần (component) có khả năng tương tác cao, hỗ trợ lưu lại trạng thái của các thành phần này cũng như có thể tái sử dụng. Trong những năm trở lại đây, ReactJS dần trở nên phổ biến trong cộng đồng lập trình viên và được sử dụng bởi nhiều công ty nổi tiếng như Netflix, Uber,... Hệ thống SSO trong bài toán này sẽ sử dụng ReactJS làm thư viện hỗ trợ xây dựng giao diện người dùng cho các trang web mặt trước của hệ thống.

1.13. Tổng kết chương

Chương 1 đã trình bày về một số khái niệm chung cũng như cơ sở lý thuyết về các công nghệ mà hệ thống sẽ sử dụng. Từ những nội dung kể trên, chương này cũng chỉ ra những đặc điểm phù hợp của các công nghệ sẽ được sử dụng để đáp ứng những yêu cầu bài toán đặt ra.

²⁰ [https://en.wikipedia.org/wiki/React_\(software\)](https://en.wikipedia.org/wiki/React_(software))

CHƯƠNG 2. PHÂN TÍCH YÊU CẦU

Trong Chương 2 này, KLTN sẽ tập trung vào khía cạnh phân tích các yêu cầu của hệ thống SSO, bắt đầu từ việc làm rõ bài toán, đối tượng sử dụng từ đó đưa ra các yêu cầu chức năng và phi chức năng của hệ thống.

2.1. Xác định bài toán và đối tượng sử dụng

Bài toán được đặt ra là cần xây dựng một hệ thống xác thực tập trung và đăng nhập một lần cho các ứng dụng và sản phẩm nội bộ của doanh nghiệp. Ngoài ra, hệ thống còn cần có luồng quy trình hỗ trợ quản lý các thông tin cá nhân, bao gồm thông tin định danh trên các hệ thống ứng dụng của cán bộ nhân viên, thêm và phê duyệt một hoặc một nhóm nhân viên mới, hỗ trợ cài đặt/thay đổi mật khẩu cũng như các phương thức xác thực MFA, phân quyền các tài khoản vào các nhóm nghiệp vụ chức năng khác nhau.

Đối với đối tượng là các cán bộ nhân viên, hệ thống phải có giao diện trực quan, dễ sử dụng, thân thiện với người dùng. Đối với Quản trị viên, hệ thống phải cung cấp được giao diện với đầy đủ các chức năng theo thẩm quyền của Quản trị viên, ngăn chặn được những thành viên không có thẩm quyền có thể thao tác làm ảnh hưởng đến dữ liệu và tính toàn vẹn trong thông tin về định danh và phân quyền của các cán bộ nhân viên khác.

2.2. Phân tích các yêu cầu của hệ thống

Các yêu cầu của hệ thống có thể phân thành các yêu cầu chức năng và các yêu cầu phi chức năng. Về các yêu cầu chức năng, dựa trên bài toán được làm rõ ở phần trước, ta có thể phân các chức năng của hệ thống thành hai nhóm chính: nhóm chức năng xác thực dành cho cán bộ nhân viên và nhóm chức năng quản trị dành cho Quản trị viên. Về các yêu cầu phi chức năng, hệ thống cần đảm bảo được các yêu cầu cơ bản về khả năng vận hành, tính khả dụng, hiệu năng và tính bảo mật.

2.2.1. Yêu cầu chức năng thêm tài khoản mới và cập nhật thông tin

Ban đầu, hệ thống sẽ chỉ tạo trước một tài khoản Quản trị viên duy nhất có tất cả các quyền trên hệ thống, bao gồm quyền tạo mới tài khoản. Hệ thống không cung cấp chức năng đăng ký tài khoản tùy chọn mà yêu cầu Quản trị viên thao tác cấp mới tài khoản cho cán bộ nhân viên thông qua giao diện của hệ thống. Khi tạo mới một tài khoản, Quản trị viên có thể thêm từng tài khoản trên giao diện hoặc tải lên một tệp chứa thông tin tài khoản cần tạo theo một mẫu được thống nhất. Sau khi tài khoản đã được tạo, Quản trị viên sẽ gửi yêu cầu đặt mật khẩu cho tài khoản cho cán bộ nhân viên và hệ thống sẽ gửi thông tin về hướng dẫn đặt mật khẩu tới email cá nhân của nhân viên một cách tự động.

Quản trị viên cũng có thể thao tác trên giao diện hệ thống để cập nhật các thông tin cá nhân của các cán bộ nhân viên. Tuy nhiên, Quản trị viên không thể trực tiếp đặt lại mật khẩu của cán bộ nhân viên khác. Khi các cán bộ nhân viên của công ty quên mật khẩu và có yêu cầu đặt lại mật khẩu, Quản trị viên có thể thao tác tạo yêu cầu đặt lại mật khẩu và hệ thống sẽ gửi thông tin về hướng dẫn đặt lại mật khẩu tới email cá nhân của nhân viên một cách tự động.

2.2.2. Yêu cầu chức năng phân quyền vào nhóm nghiệp vụ

Quản trị viên có thể tạo một nhóm nghiệp vụ mới và phân quyền cho tài khoản của cán bộ nhân viên vào nhóm nghiệp vụ này hoặc các nhóm nghiệp vụ đã có. Một nhân viên có thể được phân quyền vào nhiều nhóm nghiệp vụ khác nhau, mỗi nhóm nghiệp vụ thể hiện những quyền khác nhau của người dùng trên các hệ thống nội bộ khác nhau của doanh nghiệp.

2.2.3. Yêu cầu chức năng đăng nhập một lần

Đối với cán bộ nhân viên, khi được cấp tài khoản, cán bộ nhân viên chỉ cần đăng nhập một lần trên hệ thống xác thực SSO. Sau khi được xác thực thành công trên hệ thống SSO, người dùng có thể truy cập trực tiếp vào các ứng dụng nội bộ khác của doanh nghiệp mà không cần phải đăng nhập lại cũng như có thể thao tác trên các ứng dụng này với các quyền được chỉ định theo các nhóm nghiệp vụ mà người dùng được phân quyền. Để đảm bảo về yêu cầu an toàn thông tin và chống tấn công brute-force, hệ thống sẽ không thực hiện xác thực nếu người dùng thực hiện thao tác đăng nhập thành công quá 3 lần liên tiếp cũng như khoá chức năng đăng nhập trong vòng 10 phút nếu người dùng đăng nhập thất bại quá 5 lần liên tiếp.

2.2.4. Yêu cầu chức năng đặt/đặt lại mật khẩu

Người dùng không cần thao tác yêu cầu đặt mật khẩu. Quản trị viên sẽ chủ động tạo các yêu cầu đặt mật khẩu cho cán bộ nhân viên mới và hệ thống tự động gửi yêu cầu đặt mật khẩu tới email cá nhân của cán bộ nhân viên. Khi nhận được email hướng dẫn, cán bộ nhân viên ấn vào đường dẫn được đính kèm và thực hiện đặt mật khẩu mới cho tài khoản của mình. Thao tác đặt lại mật khẩu cũng được thực hiện tương tự. Tuy nhiên, cán bộ nhân viên cần phải thực hiện thao tác yêu cầu đặt lại mật khẩu trên hệ thống SSO bằng việc điền một số thông tin cá nhân bắt buộc để xác thực. Khi nhận được yêu cầu đặt lại mật khẩu của cán bộ nhân viên, Quản trị viên sẽ tạo yêu cầu đặt lại mật khẩu trên hệ thống để cán bộ nhân viên có thể thao tác đặt lại mật khẩu cho tài khoản của mình.

2.2.5. Các yêu cầu phi chức năng

Bài toán cũng đặt ra một số yêu cầu phi chức năng của hệ thống. Trước hết, về khả năng vận hành, hệ thống cần đảm bảo có thể cho phép tối đa 1000 người dùng có thể sử dụng và thao tác với hệ thống cùng một lúc với độ trễ không quá 5 giây trong điều kiện kết nối mạng ổn định. Hệ thống cũng cần đơn giản, dễ tương tác, tránh các tác vụ quá rườm rà. Giao diện của hệ thống cũng cần đảm bảo tương thích với các thiết bị khác nhau từ PC, laptop, máy tính bảng, thiết bị di động,... với các hệ điều hành khác nhau.

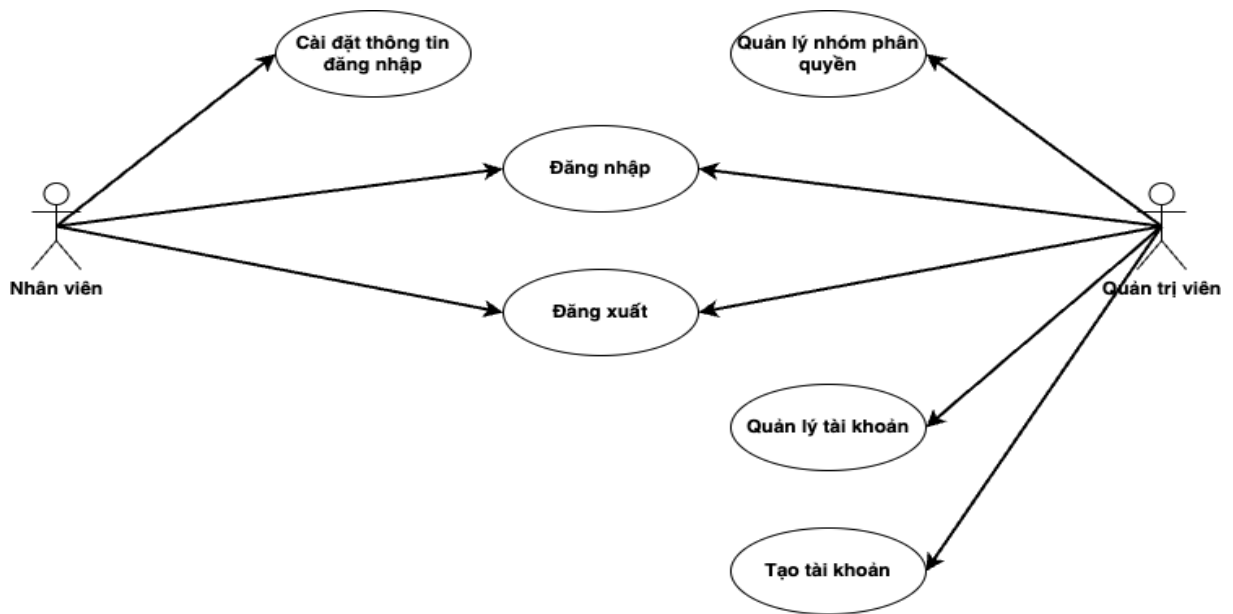
Về tính khả dụng, hệ thống cần đáp ứng được tần suất truy cập cao, thời gian đáp ứng 24/7. Thời gian khả dụng cần đạt 99% tương ứng với thời gian hệ thống ngừng hoạt động (downtime) một năm không quá 87 giờ 36 phút. Khi hệ thống ngừng vận hành do gặp sự cố, hệ thống cần đảm bảo khả năng khôi phục vận hành trong thời gian tối thiểu tránh làm gián đoạn tới hoạt động công việc của các đội nhóm nghiệp vụ.

Về tính bảo mật, hệ thống phải ngăn chặn người dùng đăng nhập vào hệ thống mà không có mật khẩu hoặc sử dụng sai mật khẩu. Hệ thống cũng cần đảm bảo chỉ có tài khoản của Quản trị viên mới được quyền thực hiện thao tác liên quan đến thêm, sửa, xóa tài khoản của cán bộ nhân viên trên hệ thống.

Về chi phí và quản lý, hệ thống cần đảm bảo mức chi phí tối thiểu trong quá trình vận hành và phát triển, không phát sinh các chi phí liên quan đến sở hữu, vận hành và bảo trì các hạ tầng phần cứng. Hệ thống có thể thu nhỏ quy mô triển khai khi lượng người dùng truy cập giảm xuống hoặc tạm dừng hoạt động trong quá trình không có người dùng sử dụng để tránh phát sinh chi phí sử dụng các dịch vụ đám mây của AWS nhưng vẫn đảm bảo khả năng mở rộng khi số lượng người sử dụng tăng cao.

2.3. Phân tích và đặc tả ca sử dụng

Với những yêu cầu chức năng kể trên, ta có thể phân các chức năng kể trên thành các ca sử dụng như trong hình bên dưới.



Hình 2.1. Biểu đồ ca sử dụng của hệ thống

2.3.1. Đặc tả ca sử dụng Đăng nhập

Mô tả:

Ca sử dụng này mô tả cách cán bộ nhân viên đăng nhập vào hệ thống SSO và sử dụng các ứng dụng khác mà không cần phải đăng nhập lại.

Luồng sự kiện:

Luồng cơ bản:

1. Cán bộ nhân viên truy cập vào ứng dụng mà mình muốn sử dụng và được điều hướng tới trang đăng nhập của hệ thống SSO hoặc truy cập trực tiếp vào trang đăng nhập của hệ thống SSO.

2. Cán bộ nhân viên nhập chính xác tên đăng nhập, mật khẩu và mã xác thực MFA trên thiết bị của mình (nếu trước đó cán bộ nhân viên có thiết lập MFA).

3. Hệ thống hiển thị cán bộ nhân viên đăng nhập thành công và chuyển hướng cán bộ nhân viên về ứng dụng ban đầu (nếu cán bộ nhân viên trước đó không truy cập trực tiếp vào trang đăng nhập của hệ thống SSO).

Luồng ngoại lệ:

Nếu cán bộ nhân viên nhập sai tài khoản, mật khẩu hoặc mã MFA, hệ thống hiển thị lỗi trên giao diện của trang đăng nhập.

Yêu cầu đặc biệt:

Không có.

Tiền điều kiện:

Không có.

Hậu điều kiện:

Người dùng được chuyển hướng tới các ứng dụng người dùng muốn truy cập hoặc khi người dùng đăng nhập vào các ứng dụng trong hệ thống nội bộ trong thời gian hiệu lực của phiên đăng nhập, người dùng không cần phải thực hiện lại quá trình đăng nhập.

Điểm mở rộng:

Không có.

2.3.2. Đặc tả ca sử dụng Tạo tài khoản

Mô tả:

Ca sử dụng này mô tả cách Quản trị viên của hệ thống SSO tạo tài khoản cho nhân viên mới gia nhập doanh nghiệp.

Luồng sự kiện:

Luồng cơ bản:

1. Quản trị viên truy cập vào giao diện tạo tài khoản mới.
2. Quản trị viên có thể thêm người dùng mới bằng cách điền trực tiếp thông tin tài khoản cần tạo mới vào các ô thông tin trên giao diện hoặc hoặc tải tệp dữ liệu chứa thông tin tài khoản mới lên hệ thống.
3. Quản trị viên kiểm tra lại các tài khoản mới sẽ được thêm vào hệ thống và xác nhận thêm tài khoản mới.
4. Quản trị viên chọn các tài khoản mới tại giao diện hiển thị kết quả tạo tài khoản và ấn nút yêu cầu gửi email đặt mật khẩu mới cho tài khoản.

Luồng ngoại lệ:

Khi tạo mới tài khoản, nếu tên đăng nhập trùng với tên đăng nhập đã có trên hệ thống, hệ thống sẽ hiển thị lỗi liệt kê (các) tên đăng nhập phát sinh lỗi trong quá trình tạo tài khoản.

Yêu cầu đặc biệt:

Không có.

Tiền điều kiện:

Quản trị viên đăng nhập vào hệ thống bằng tài khoản có quyền Quản trị viên hệ thống.

Hậu điều kiện:

Hệ thống tạo tài khoản người dùng mới theo đúng các thông tin Quản trị viên cung cấp và lưu vào Amazon Cognito. Người dùng được tạo nhận được email yêu cầu đặt mật khẩu với hướng dẫn về đường dẫn thực hiện.

Điểm mở rộng:

Không có.

2.3.3. Đặc tả ca sử dụng Thiết lập thông tin đăng nhập

Mô tả:

Ca sử dụng này mô tả cách người dùng thiết lập thông tin đăng nhập (bao gồm mật khẩu và thiết lập phương thức xác thực MFA) cho tài khoản của mình.

Luồng sự kiện:

Luồng cơ bản:

1. Khi cán bộ nhân viên nhận được email yêu cầu thiết lập/đặt lại thông tin đăng nhập, người dùng truy cập theo đường dẫn được cung cấp trong email.
2. Để thiết lập mật khẩu mới, cán bộ nhân viên nhập mật khẩu mới và mật khẩu cũ (nếu có) cùng với thông tin về email.
3. Để thiết lập phương thức xác thực MFA, cán bộ nhân viên sử dụng các phần mềm xác thực MFA (ví dụ như Google Authenticator) thiết bị di động của mình quét mã QR hiển thị trên màn hình giao diện hệ thống.
4. Để thiết lập phương thức xác thực MFA, sau khi quét mã QR thành công, cán bộ nhân viên nhập mã MFA gồm 6 chữ số hiển thị trên giao diện của phần mềm xác thực MFA.
5. Cán bộ nhân viên ấn nút xác nhận thực hiện đặt mật khẩu mới.

Luồng ngoại lệ:

1. Nếu mật khẩu cũ và mật khẩu mới trùng nhau hệ thống hiển thị lỗi.
2. Nếu mật khẩu cũ, email, tên đăng nhập không chính xác/đường dẫn để thực hiện yêu cầu thay đổi mật khẩu không đúng so với đường dẫn được cung cấp bởi Quản trị viên, hiển thị lỗi trên giao diện hệ thống.

Yêu cầu đặc biệt:

Không có.

Tiền điều kiện:

Quản trị viên thực hiện thao tác yêu cầu đặt lại mật khẩu cho người dùng trên hệ thống với tài khoản Quản trị viên.

Hậu điều kiện:

Thông tin mật khẩu của người dùng được cập nhật trên Amazon Cognito, mã xác thực và phiên đăng nhập của người dùng (nếu có) được lưu trên hệ thống bị vô hiệu hoá.

Điểm mở rộng:

Không có.

2.3.4. Đặc tả ca sử dụng Quản lý tài khoản**Mô tả:**

Ca sử dụng này mô tả cách Quản trị viên của hệ thống SSO quản lý tài khoản định danh của cán bộ nhân viên của công ty.

Luồng sự kiện:*Luồng cơ bản:*

1. Quản trị viên truy cập vào giao diện quản lý tài khoản.
2. Quản trị viên có thể tìm kiếm người dùng bằng thanh tìm kiếm hoặc bấm chọn vào tên người dùng hiển thị trên giao diện.
3. Quản trị viên có thể sửa đổi các thông tin cá nhân của người dùng (không thể thay đổi tên đăng nhập, email và mật khẩu).
4. Quản trị viên có thể lựa chọn xoá người dùng khỏi hệ thống.
5. Quản trị viên có thể yêu cầu người dùng thay đổi mật khẩu bằng việc ấn nút yêu cầu thay đổi mật khẩu.
6. Quản trị viên ấn nút để xác nhận các thao tác vừa thực hiện.

Luồng ngoại lệ:

Giao diện hệ thống hiển thị kết quả thực hiện yêu cầu của Quản trị viên (bao gồm kết quả thành công và kết quả lỗi).

Yêu cầu đặc biệt:

Không có.

Tiền điều kiện:

Quản trị viên đăng nhập vào hệ thống bằng tài khoản có quyền Quản trị viên hệ thống.

Hậu điều kiện:

Hệ thống cập nhật tài khoản người dùng mới theo đúng các thông tin Quản trị viên cung cấp và lưu vào Amazon Cognito. Người dùng được tạo nhận được email yêu cầu

đặt lại mật khẩu với hướng dẫn về đường dẫn thực hiện nếu thao tác được yêu cầu là đặt lại mật khẩu.

Điểm mở rộng:

Không có.

2.3.5. Đặc tả ca sử dụng Quản lý nhóm phân quyền

Mô tả:

Ca sử dụng này mô tả cách Quản trị viên tạo các nhóm phân quyền trên hệ thống SSO cũng như quản lý các thành viên trong các nhóm phân quyền này.

Luồng sự kiện:

Luồng cơ bản:

1. Quản trị viên truy cập giao diện quản lý nhóm phân quyền.
2. Quản trị viên có thể tạo thêm nhóm phân quyền mới bằng cách ấn nút tạo thêm nhóm phân quyền.
3. Quản trị viên có thể chọn một nhóm phân quyền trong danh sách nhóm phân quyền và thêm một/một nhóm tài khoản vào nhóm phân quyền.
4. Quản trị viên cũng có thể loại bỏ một tài khoản khỏi danh sách thành viên của nhóm phân quyền.
5. Quản trị viên ấn nút xác nhận thực hiện thao tác.

Luồng ngoại lệ:

Giao diện hệ thống hiển thị kết quả thực hiện yêu cầu của quản trị viên (bao gồm kết quả thành công và kết quả lỗi).

Yêu cầu đặc biệt:

Không có.

Tiền điều kiện:

Quản trị viên đăng nhập vào hệ thống bằng tài khoản có quyền Quản trị viên hệ thống.

Hậu điều kiện:

Hệ thống thực hiện cập nhật thông tin vào Amazon Cognito cũng như vô hiệu hoá mã xác thực và phiên đăng nhập của người dùng (nếu có).

Điểm mở rộng:

Không có.

2.3.6. Đặc tả ca sử dụng Đăng xuất

Mô tả:

Ca sử dụng này mô tả cách cán bộ nhân viên thực hiện đăng xuất khỏi tài khoản của mình trên các ứng dụng hoặc trên hệ thống SSO.

Luồng sự kiện:

Luồng cơ bản:

1. Tại màn hình ứng dụng hoặc tại màn hình hệ thống SSO, người dùng lựa chọn nút đăng xuất.

2. Hệ thống thực hiện vô hiệu hoá phiên đăng nhập của cán bộ nhân viên trên tất cả các ứng dụng trong hệ thống nội bộ và điều hướng cán bộ nhân viên tới trang đăng nhập của hệ thống SSO.

Luồng ngoại lệ:

Không có.

Yêu cầu đặc biệt:

Không có.

Tiền điều kiện:

Cán bộ nhân viên đã đăng nhập thành công trên hệ thống SSO và vẫn trong thời gian hiệu lực của phiên đăng nhập đó.

Hậu điều kiện:

Cán bộ nhân viên đăng xuất ra khỏi tất cả các ứng dụng. Khi người dùng truy cập vào hệ thống SSO hoặc bất kỳ ứng dụng khác sẽ phải thực hiện đăng nhập lại.

Điểm mở rộng:

Không có.

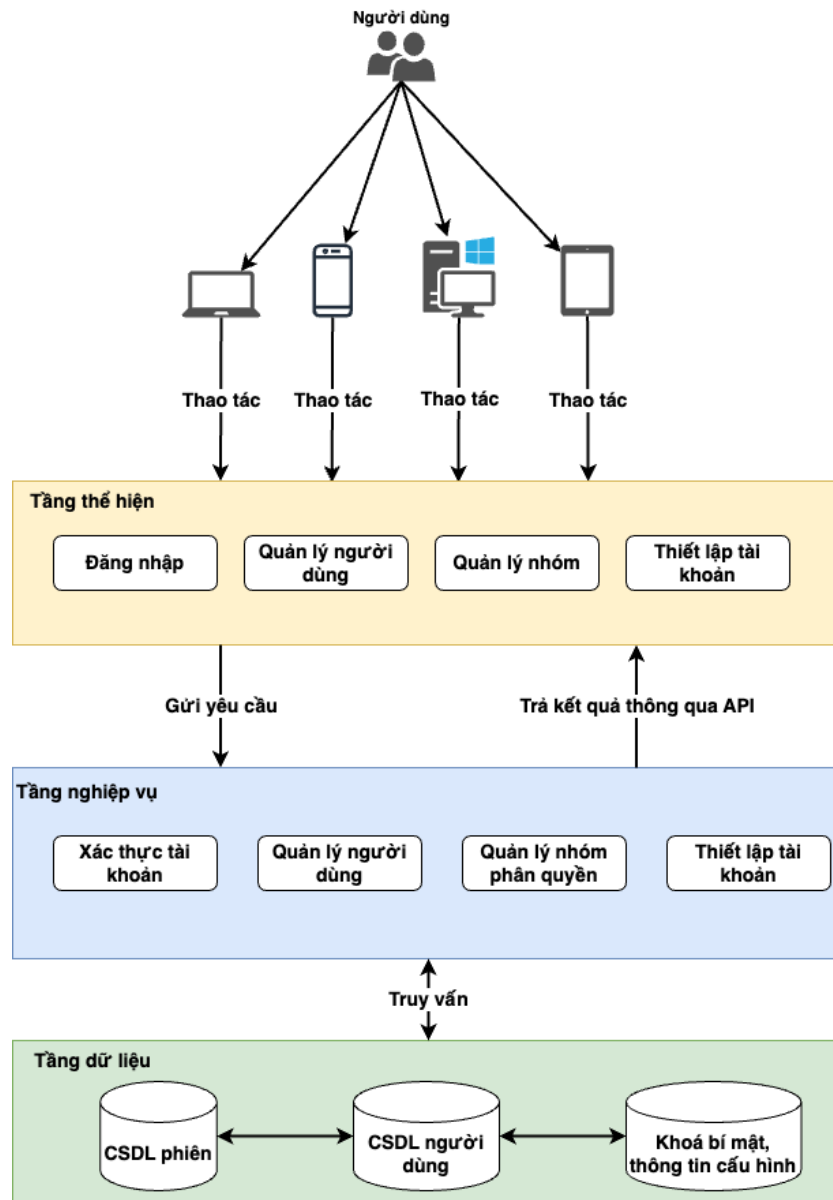
2.4. Tổng kết chương

Trong chương này, KLTN đã trình bày một cách chi tiết về yêu cầu của bài toán được đặt ra ở phần đầu của KLTN cũng như tập trung phân tích các yêu cầu chức năng và phi chức năng của hệ thống SSO phi máy chủ. Chương 2 cũng đã phân tích và đặc tả các ca sử dụng chính của hệ thống. Từ những yêu cầu và đặc điểm như vậy, chương tiếp theo sẽ tiếp tục trình bày về mô hình thiết kế và xây dựng hệ thống, cách hệ thống sử dụng kết hợp các dịch vụ được quản lý và phi máy chủ của AWS để có thể tận dụng sức mạnh của các dịch vụ đám mây được cung cấp bởi AWS, cũng như hiện thực hoá các ca sử dụng thành các luồng hoạt động của hệ thống.

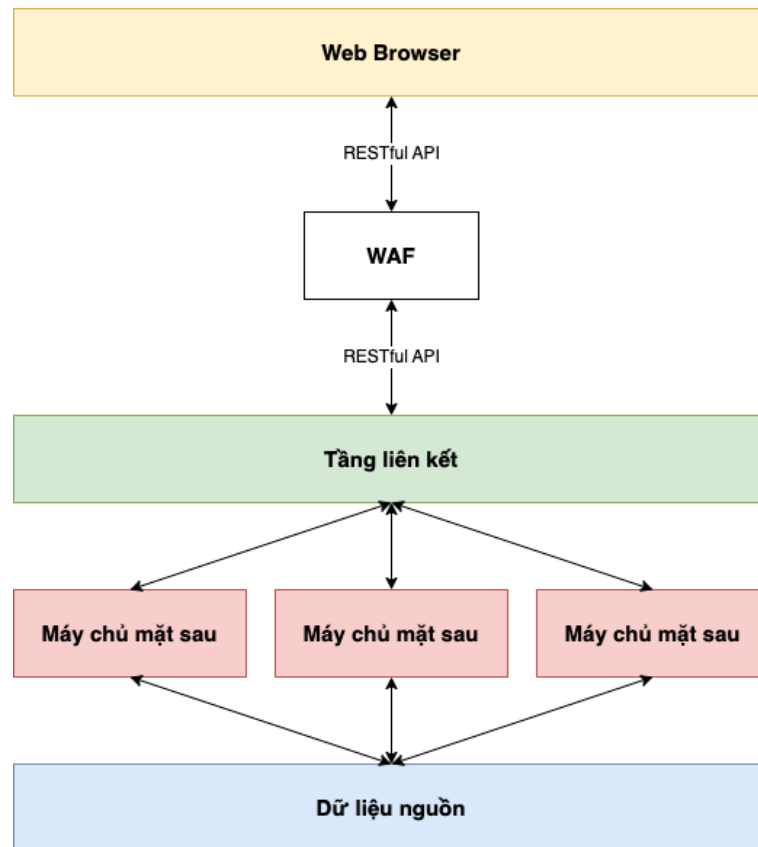
CHƯƠNG 3. THIẾT KẾ HỆ THỐNG

Trong Chương 3, KLTN sẽ trình bày về mô hình thiết kế của **Hệ thống SSO phi máy chủ** cũng như cách triển khai các dịch vụ đám mây của AWS đã trình bày ở Chương 1 trong mô hình kiến trúc của hệ thống. Bên cạnh đó, chương này cũng mô tả các luồng hoạt động của hệ thống dựa trên các ca sử dụng đã mô tả ở Chương 2.

3.1. Tổng quan thiết kế hệ thống



Hình 3.1. Kiến trúc tổng quan của hệ thống



Hình 3.2. Mô hình kiến trúc mức cao

Ta xây dựng kiến trúc của hệ thống bao gồm ba tầng: tầng dữ liệu (Data Layer), tầng nghiệp vụ (Business Layer), tầng thể hiện (Presentation Layer) như Hình 3.1. Từ đó, ta cũng mô hình hoá kiến trúc mức cao của hệ thống như Hình 3.2.

Tầng dữ liệu có thể nói là tầng thiết yếu của bất cứ một hệ thống nào. Tầng này đảm nhiệm việc lưu trữ, ghi, đọc, sửa, xóa cơ sở dữ liệu. Với ngữ cảnh của KLTN này, tầng dữ liệu đảm nhận việc quản lý và lưu trữ ba loại thông tin là dữ liệu về phiên, dữ liệu người dùng cũng như các dữ liệu liên quan đến khoá bí mật và thông tin cấu hình. Tầng dữ liệu của hệ thống sẽ tận dụng các dịch vụ lưu trữ đám mây của AWS đó là Amazon Cognito, Amazon DynamoDB. Như đã giới thiệu ở Chương 1, ta sử dụng Amazon Cognito làm CSDL lưu trữ thông tin đăng nhập cũng như các thông tin cá nhân của nhân viên cũng như hỗ trợ khả năng xác thực người dùng. Ta cũng sử dụng CSDL phi quan hệ Amazon DynamoDB để lưu trữ thông tin các phiên đăng nhập cũng như thông tin về các yêu cầu thiết lập thông tin đăng nhập từ Quản trị viên. Bên cạnh hai dịch vụ đã được giới thiệu kể trên, hệ thống còn sử dụng thêm hai dịch vụ lưu trữ đám mây khác là chức năng Parameter Store²¹ của AWS Systems Manager cũng như AWS

²¹ <https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-parameter-store.html>

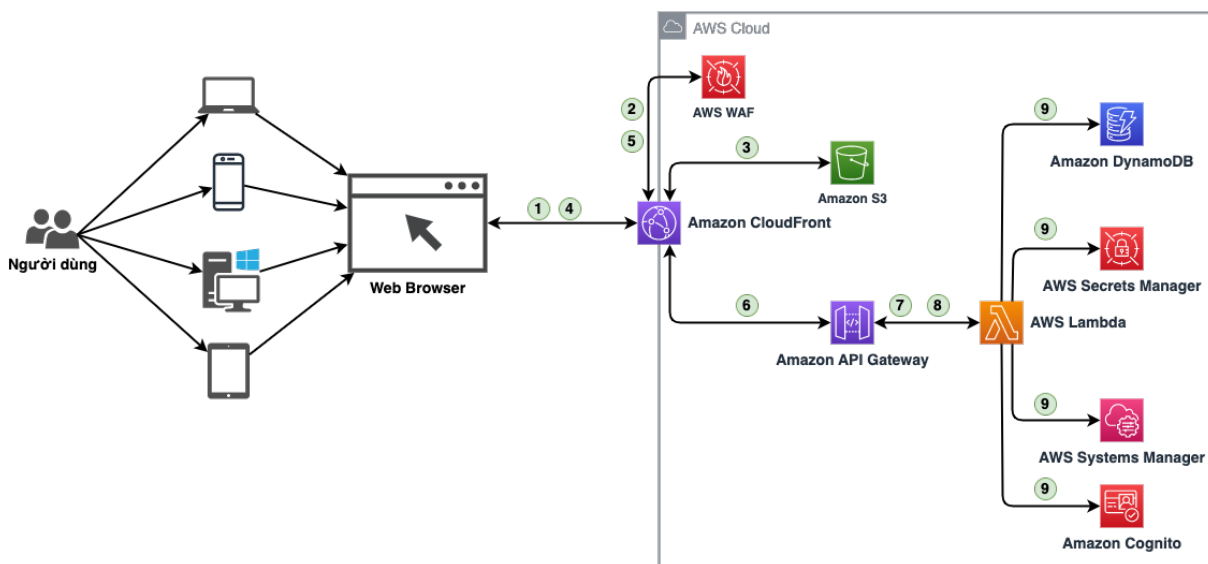
Secrets Manager²². Hai dịch vụ này được sử dụng để lưu trữ các thông tin có độ yêu cầu bảo mật mà ta không muốn lưu trữ trực tiếp trong mã nguồn hay các biến môi trường của Lambda ví dụ như các khoá (key) xác thực, các thông tin về cấu hình,...

Đối với tầng nghiệp vụ, đây chính là nơi có nhiệm vụ xử lý các vấn đề về mặt logic, thuật toán, truy vấn cũng như tương tác trực tiếp với tầng dữ liệu. Trong bài toán đặt ra ở KLTN này, ta sử dụng dịch vụ phi máy chủ của AWS là AWS Lambda để xử lý các yêu cầu về mặt nghiệp vụ như xác thực thông tin đăng nhập, kiểm tra xác thực, thiết lập thông tin đăng nhập, quản lý người dùng và nhóm phân quyền,...

Tầng thể hiện bao gồm các thiết bị của người dùng cuối như máy tính, thiết bị di động,... Tầng này quản lý, phân phối nội dung và hiển thị đến người dùng. Người dùng có thể thực hiện các yêu cầu chức năng và gửi yêu cầu đến tầng nghiệp vụ xử lý thông qua các RESTful API. Ở tầng này, ta sử dụng Amazon S3 để cung cấp các trang web tĩnh và phân phối nội dung các trang web này tới thiết bị của người dùng thông qua Amazon CloudFront. Tầng này cũng giao tiếp với tầng nghiệp vụ bằng API thông qua Amazon API Gateway. Để chặn các tấn công ngoài mong muốn cũng như đảm bảo an toàn cho hệ thống, ta sử dụng thêm một dịch vụ đám mây khác là AWS WAF.

3.2. Mô tả luồng tổng quan của hệ thống

Từ sơ đồ thiết kế đã được trình bày trên, ta có thể mô tả luồng hoạt động tổng quan của hệ thống như hình vẽ sau (Hình 3.3), cụ thể:



Hình 3.3. Mô tả luồng tổng quan của hệ thống

²² <https://docs.aws.amazon.com/secretsmanager/latest/userguide/intro.html>

1. Người dùng truy cập hệ thống thông qua các trình duyệt web trên thiết bị của mình. Tên miền của hệ thống được các hệ thống tên miền phân giải và chuyển tiếp yêu cầu tới vị trí biên của CloudFront gần nhất với người dùng (ví dụ vị trí biên của AWS tại Hà Nội).

2. AWS WAF được gắn trực tiếp với CloudFront kiểm tra truy vấn của người dùng và chặn các yêu cầu không hợp lệ hoặc có dấu hiệu tấn công. Nếu truy vấn hợp lệ, truy vấn tiếp tục được chuyển tiếp xử lý tại CloudFront.

3. Amazon CloudFront kiểm tra xem có bản ghi trong bộ nhớ đệm của dữ liệu tĩnh (trang web tĩnh, hình ảnh,...) cần truy vấn hay không, nếu có lấy dữ liệu bộ nhớ đệm và trả về người dùng. Nếu các trung tâm dữ liệu tại vị trí biên của CloudFront chưa có các bản ghi bộ nhớ đệm của dữ liệu thì sẽ truy vấn đến dữ liệu nguồn tại Amazon S3 tại các khu vực của AWS (ví dụ Singapore) và trả về cho người dùng cũng như lưu dữ liệu bộ nhớ đệm cho những lần truy vấn sau.

4. Người dùng thực hiện các thao tác trên giao diện của hệ thống thông qua các trình duyệt web. Giao diện mặt trước của hệ thống gửi các yêu cầu API đến CloudFront.

5. Tương tự bước 2, AWS WAF kiểm tra các yêu cầu truy vấn của người dùng và ngăn chặn các truy vấn không hợp lệ hoặc khả nghi.

6. CloudFront chuyển tiếp các truy vấn RESTful API đến API Gateway.

7. API Gateway truy vấn các đường dẫn tài nguyên đã được cấu hình. Nếu truy vấn có đường dẫn tài nguyên phù hợp, API Gateway sẽ chuyển yêu cầu truy vấn đến một hàm Lambda đặc biệt có nhiệm vụ xác thực truy vấn (đối với các đường dẫn tài nguyên có yêu cầu xác thực truy vấn, ví dụ các đường dẫn dành cho Quản trị viên yêu cầu có mã xác thực với nhóm phân quyền thuộc nhóm Quản trị viên). Hàm Lambda xác thực truy vấn và trả về kết quả người dùng có được phân quyền thực hiện yêu cầu hay không.

8. Nếu người dùng được phân quyền thực hiện yêu cầu, API Gateway sẽ gọi tới hàm Lambda được cấu hình tương ứng với đường dẫn tài nguyên mà người dùng truy vấn.

9. AWS Lambda khởi tạo môi trường thực thi khi nhận được sự kiện từ API Gateway, truy vấn các dữ liệu từ Amazon Cognito, Amazon DynamoDB,... và thực hiện các yêu cầu của người dùng cũng như trả về kết quả thực hiện cho API Gateway để trả về cho người dùng thông qua CloudFront.

3.3. Thiết kế tầng dữ liệu

Phần này sẽ giới thiệu chi tiết hơn về một số đặc điểm trong việc thiết kế và xây dựng tầng dữ liệu của hệ thống. Như đã đề cập ở phần trước, ta có ba loại dữ liệu cần thực hiện lưu trữ: dữ liệu phiên, dữ liệu người dùng và dữ liệu cấu hình, khoá bí mật.

Trước hết nói về dữ liệu người dùng, Amazon Cognito lưu trữ dữ liệu người dùng thành hai nhóm thông tin chính là thông tin người dùng và thông tin về nhóm phân quyền. Đối với thông tin nhóm phân quyền, Cognito cung cấp khả năng lưu trữ, quản lý và truy vấn danh sách các nhóm phân quyền cũng như danh sách thành viên của từng nhóm phân quyền. Bên cạnh đó, Cognito hỗ trợ lưu trữ, quản lý và truy vấn các thông tin về cá nhân của người dùng (gồm các thông tin cơ bản được định nghĩa trước và thông tin tùy biến) cũng như thông tin về các nhóm phân quyền của người dùng đó. Tầng nghiệp vụ sẽ giao tiếp với tầng dữ liệu thông qua một bộ công cụ phát triển phần mềm (SDK) của AWS là boto3²³.

Dữ liệu được lưu trữ trong Cognito về thông tin của người dùng cũng như các thông tin liên quan tới các nhóm phân quyền được thể hiện trong Hình 3.4, 3.5 cũng như Bảng 3.1, 3.2 bên dưới.

Group information					Edit
Group name	Description	Created time			
IT_SystemAdministrator	Group for System Administrator of SSO System.	March 19, 2023 at 00:06 GMT+7			
IAM Role ARN	Precedence	Last updated time			
-	1	April 13, 2023 at 16:31 GMT+7			

Group members (1) Info					Remove user from group	Add user to group
< 1 > ⚙						
User name	Email address	Email verified	Confirmation status	Status		
☐ datnt3.ho		Yes	Confirmed	Ena		

Hình 3.4. Dữ liệu về nhóm phân quyền lưu trữ trên Amazon Cognito

²³ <https://aws.amazon.com/vi/sdk-for-python/>

Để tăng nghiệp vụ có thể giao tiếp với Cognito thông qua boto3 nhằm xử lý các tác vụ liên quan đến nhóm phân quyền, tăng nghiệp vụ có thể sử dụng các thông tin sau:

Thông tin	Mô tả	Kiểu dữ liệu
GroupName	Tên của nhóm phân quyền, duy nhất trong một User Pool	string
Description	Mô tả về chức năng và quyền hạn của nhóm phân quyền (nếu có)	string
Precedence	Độ ưu tiên của nhóm phân quyền (độ ưu tiên lớn nhất là 0, giá trị càng nhỏ, độ ưu tiên càng cao)	integer
RoleArn	ARN của quyền được chỉ định tại dịch vụ AWS IAM ²⁴ , được sử dụng để phân quyền vào các dịch vụ của AWS (nếu có)	string
CreationDate	Thời gian tạo nhóm phân quyền	datetime
LastModifiedDate	Thời gian cập nhật cuối cùng của nhóm phân quyền	datetime

Bảng 3.1. Các thông tin về nhóm phân quyền

²⁴ <https://aws.amazon.com/iam/>

User information

User ID (Sub)

 cbe59479-1b15-405b-a045-20b0244b4b7c

Alias attributes used to sign in

User name

MFA setting

MFA inactive

MFA methods

-

Account status

 Enabled

Confirmation status

Confirmed

Created time

March 19, 2023 at 00:01 GMT+7

Last updated time

April 13, 2023 at 16:22 GMT+7

User attributes (12) [Info](#)

View and edit this user's attributes.

Q

Filter by property or value

<

1

>

⚙

birthdate	01/09/2001	Required
custom:department	IT	Optional
custom:personal_email	dat192001@gmail.com	Optional
custom:staff_id	02-5985	Optional
custom:title	Dev/SysOps Officer	Optional
email	Verified	Required
family_name	Nguyen Tien	Required
gender	male	Optional
given_name	Dat	Required
phone_number	+84343669258 Verified	Required
sub	cbe59479-1b15-405b-a045-20b0244b4b7c	Required

Group memberships (2) [Info](#)

View and edit this user's group memberships.

Q

Filter groups by property or value

<

1

>

⚙

Group name ▲	Description ▼	Group created time ▼
☐ IT_Member	A Group for all members from IT department.	Last week
☐ IT_SystemAdministrator	Group for System Administrator of SSO System.	Last month

Hình 3.5. Dữ liệu về người dùng lưu trữ trên Amazon Cognito

Để tăng nghiệp vụ có thể giao tiếp với Cognito thông qua boto3 nhằm xử lý các tác vụ liên quan đến thông tin người dùng, tăng nghiệp vụ có thể sử dụng các thông tin sau:

Thông tin	Mô tả	Kiểu dữ liệu
Username	Tên đăng nhập, duy nhất trong một User Pool	string
Address	Địa chỉ của nhân viên	string
Birthdate	Ngày tháng năm sinh của nhân viên	datetime
Email	Địa chỉ email của nhân viên được công ty cấp	string
FamilyName	Họ và tên đệm của nhân viên	string
GivenName	Tên của nhân viên	string
PhoneNumber	Số điện thoại của nhân viên	string
Gender	Giới tính của nhân viên	string
Department	Tên phòng ban làm việc của nhân viên	string
Title	Tên chức vụ của nhân viên	string
StaffId	Mã số nhân viên	string
PersonalEmail	Email cá nhân của nhân viên	string
Sub	Mã định danh của người dùng trong User Pool	string

Bảng 3.2. Các thông tin về người dùng

Bên cạnh các dữ liệu về người dùng lưu trữ trên Cognito, ta còn có các dữ liệu liên quan tới phiên. Các phiên được lưu trữ có thể là phiên đăng nhập hoặc thông tin về phiên yêu cầu cài đặt thông tin đăng nhập của người dùng. Các thông tin được lưu trữ trên DynamoDB gồm 3 trường chính, như mô tả trong Bảng 3.3 và Hình 3.6.

Thông tin	Mô tả	Kiểu dữ liệu
KeyID	Mã số duy nhất để định danh trường thông tin	string
Digest	Dữ liệu thực tế được lưu trữ	string
Expiration	Thời gian epoch dữ liệu hết hạn	int

Bảng 3.3. Mô tả thông tin lưu trữ trên DynamoDB

DynamoDB > Items > dev-uet-sso-dynamodb-cache-storage

dev-uet-sso-dynamodb-cache-storage

Autopreview View table details

▶ Scan or query items
Expand to query or scan items.

Items returned (2)

Actions Create item

	KeyID	Digest	Expiration (TTL)
☐	bae22c6dc4943d0a71cba99350dbedd864f68543c69bf6411097916900a640f873...	43782eba9a6b165fe056f1fc603...	1681419109
☐	bbee1b1742146d2dca6b2199004137233266b64bc3c81ef4f464da543d9638dc	https://d2xgbj4u54ra14.cloudfro...	1681494498

Hình 3.6. Dữ liệu được lưu trữ tại Amazon DynamoDB

Như đã đề cập ở phần trước, bên cạnh sử dụng Cognito và DynamoDB, hệ thống còn sử dụng thêm chức năng Parameter Store của AWS Systems Manager và AWS Secrets Manager để lưu trữ một số thông tin khác liên quan tới cấu hình và khoá bí mật như Hình 3.7.

My parameters

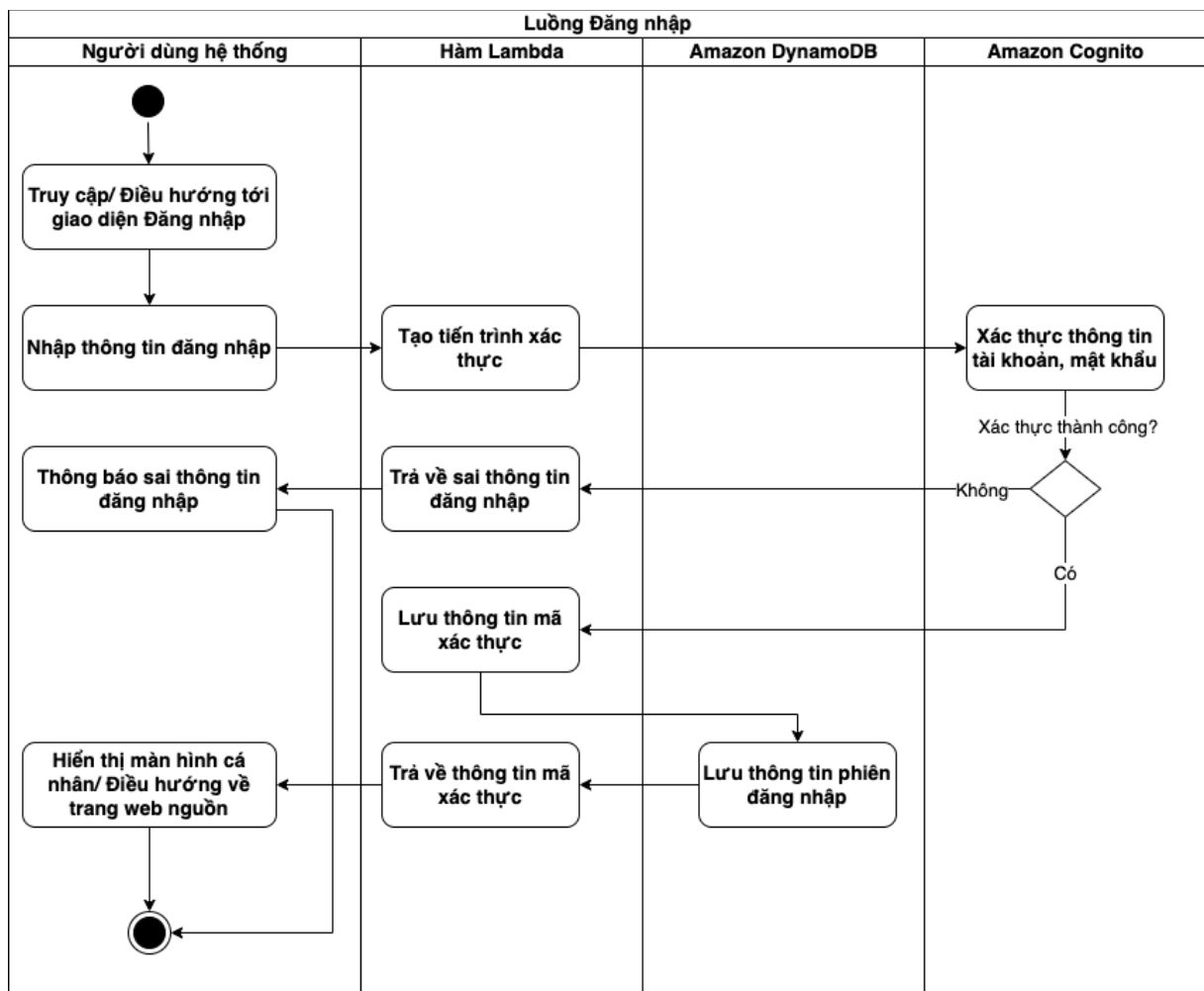
View details Edit Delete Create parameter

Search

	Name	Tier	Type	Last modified
☐	/dev/sso/API_GW_KEY	Standard	String	Sun, 09 Apr 2023 03:55:49 GMT
☐	/dev/sso/CF_DOMAIN	Standard	String	Sat, 18 Mar 2023 11:07:07 GMT
☐	/dev/sso/USER_POOL_ID	Standard	String	Sun, 19 Mar 2023 03:53:45 GMT

Hình 3.7. Một số dữ liệu lưu trữ trên Parameter Store

3.4. Thiết kế luồng Đăng nhập

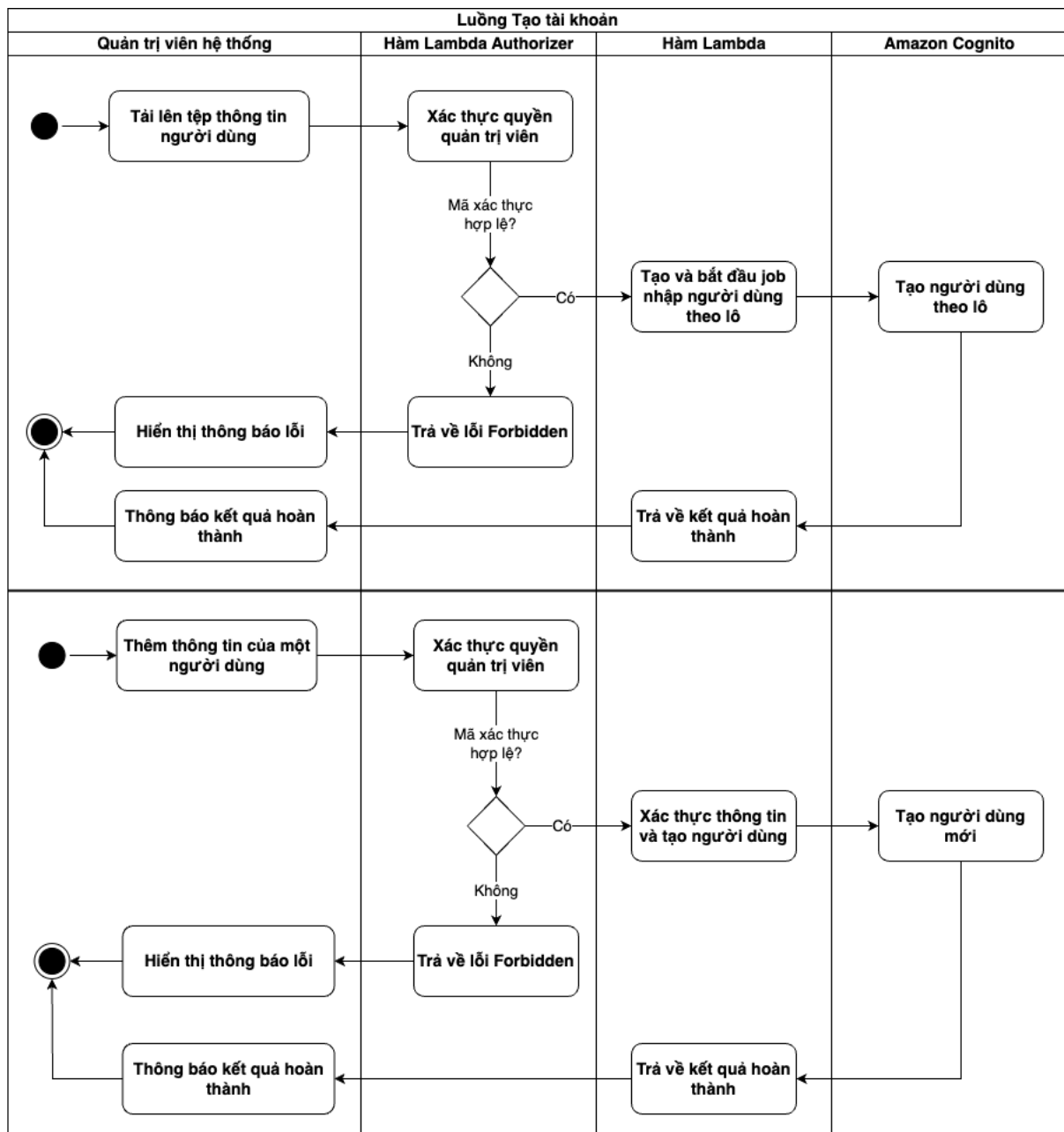


Hình 3.8. Biểu đồ luồng Đăng nhập

Hình 3.8 mô tả biểu đồ luồng cho ca sử dụng người dùng đăng nhập. Người dùng có thể bắt đầu tiến trình đăng nhập bằng việc truy cập trực tiếp vào trang đăng nhập của hệ thống SSO hoặc truy cập vào một trang web thuộc hệ thống nội bộ của doanh nghiệp và được điều hướng tới giao diện đăng nhập của hệ thống SSO. Cán bộ nhân viên tiến hành nhập các thông tin đăng nhập bao gồm tên đăng nhập, mật khẩu (và mã xác thực đa yếu tố trên thiết bị cá nhân cùng với mã captcha, nếu có). Yêu cầu đăng nhập được gửi tới hàm Lambda xử lý tác vụ xác thực. Hàm này lấy ra các thông tin đăng nhập trong yêu cầu và tạo tiến trình xác thực với Amazon Cognito. Nếu tiến trình xác thực thất bại, hàm Lambda trả về thông báo lỗi và hệ thống đăng nhập hiển thị lỗi cho người dùng. Nếu tiến trình xác thực thành công, hàm Lambda lưu trữ thông tin phiên đăng nhập vào Amazon DynamoDB cũng như trả về thông tin mã xác thực cho người dùng. Trang web đăng nhập sau khi nhận được mã xác thực tiến hành lưu lại mã xác thực vào bộ nhớ cục bộ và điều hướng đến trang cá nhân của người dùng trên hệ thống SSO hoặc điều hướng

về trang web nguồn nêu trước đó người dùng được điều hướng tới trang đăng nhập SSO từ trang web nguồn.

3.5. Thiết kế luồng Tạo tài khoản



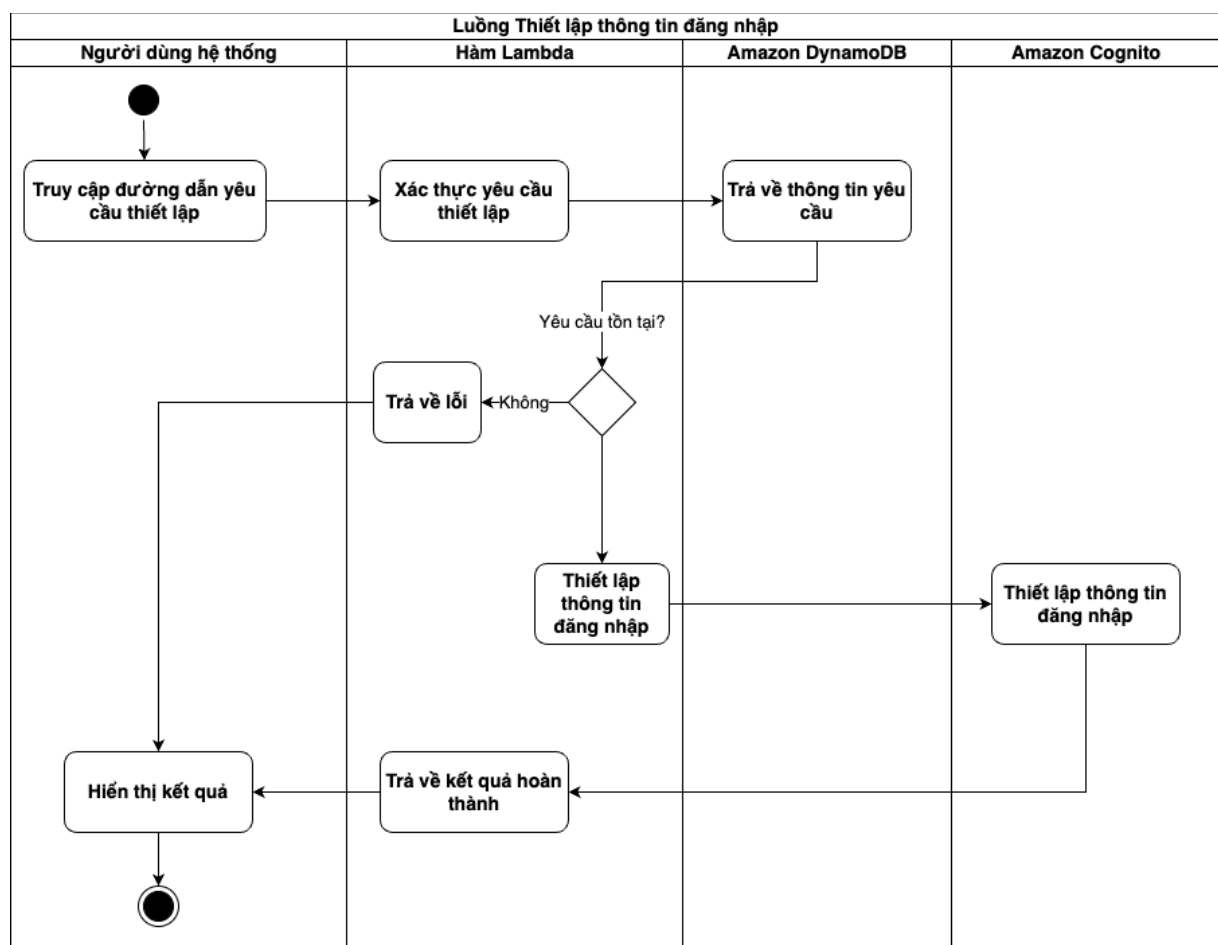
Hình 3.9. Biểu đồ luồng Tao tài khoản

Như mô tả trong biểu đồ luồng Tạo tài khoản ở Hình 3.9, Quản trị viên có thể lựa chọn tạo tài khoản theo lô hoặc tạo tài khoản đơn lẻ. Để tạo tài khoản theo lô, Quản trị viên tiến hành tải lên tập tin định dạng giá trị phân cách bằng dấu phẩy (.csv). Hệ thống sử dụng hàm Lambda Authorizer để xác thực quyền của yêu cầu. Hàm này đóng vai trò như một phần mềm trung gian được kết nối với API Gateway để thực hiện nhiệm vụ

tiền xử lý trước khi API Gateway cho các yêu cầu được tiếp tục xử lý ở các hàm Lambda xử lý nghiệp vụ. Hàm Lambda Authorizer kiểm tra quyền cũng như hiệu lực của mã xác thực được truyền vào trong yêu cầu. Nếu mã xác thực không hợp lệ sẽ chặn yêu cầu xử lý và trả về lỗi “403 – Forbidden”. Nếu mã xác thực hợp lệ, yêu cầu được chuyển tiếp tới hàm Lambda xử lý. Hàm Lambda tiến hành tạo tác vụ yêu cầu nhập người dùng theo lô tới Amazon Cognito cũng như tiến hành bắt đầu tác vụ này. Cognito nhận được yêu cầu tiến hành thêm các người dùng hợp lệ trong tệp đính kèm. Hàm Lambda trả về kết quả thực hiện và hiển thị kết quả thành công để Quản trị viên tiến hành yêu cầu người dùng đặt mật khẩu cho tài khoản.

Để tạo tài khoản đơn lẻ, Quản trị viên nhập lần lượt các trường thông tin đăng nhập cũng như thông tin cá nhân của người dùng và yêu cầu thêm người dùng. Tương tự, hàm Lambda Authorizer tiếp tục tiến hành xác thực quyền của yêu cầu. Nếu yêu cầu hợp lệ, yêu cầu chuyển tiếp đến hàm Lambda xử lý, tạo yêu cầu thêm người dùng tới Cognito và trả kết quả về cho Quản trị viên.

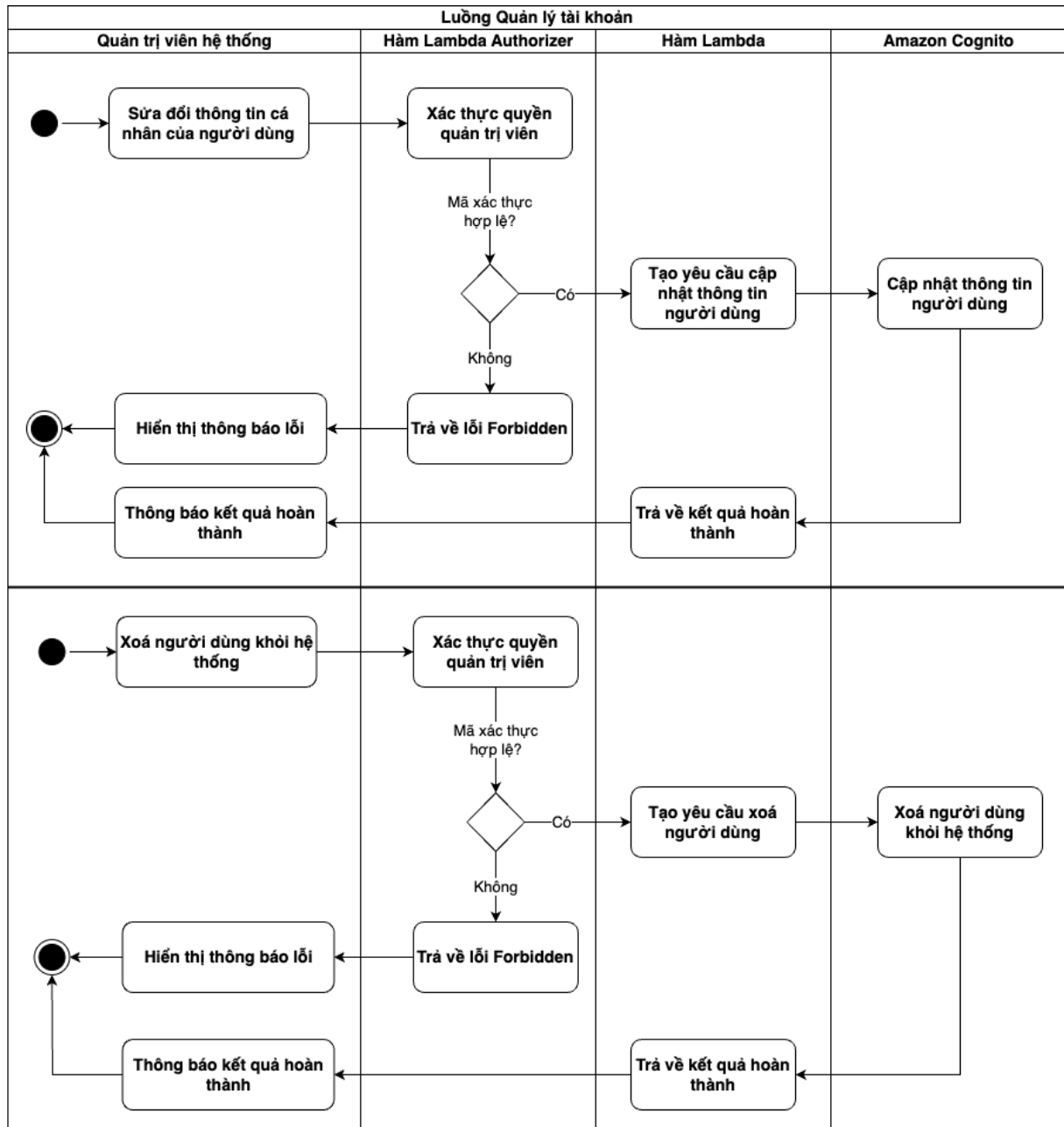
3.6. Thiết kế luồng Thiết lập thông tin đăng nhập



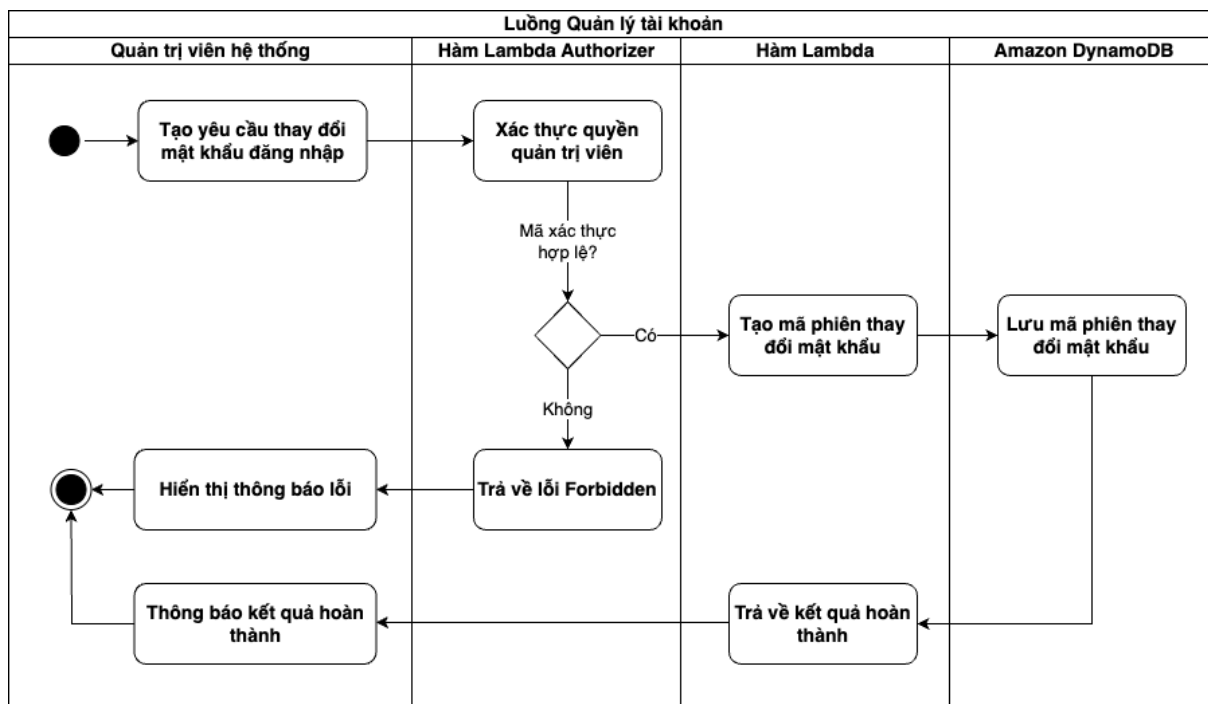
Hình 3.10. Biểu đồ luồng Thiết lập thông tin đăng nhập

Đối với luồng Thiết lập thông tin đăng nhập (Hình 3.10), khi người dùng nhận được email yêu cầu thiết lập đăng nhập, người dùng truy cập theo đường dẫn và tiến hành thiết lập thông tin đăng nhập. Hàm Lambda kiểm tra phiên yêu cầu thiết lập thông tin đăng nhập được lưu trong DynamoDB, nếu không tồn tại yêu cầu trả về lỗi yêu cầu không tồn tại. Nếu tồn tại yêu cầu, hàm Lambda tạo yêu cầu thiết lập thông tin đăng nhập tới Amazon Cognito và trả về kết quả thực hiện thiết lập cho người dùng.

3.7. Thiết kế luồng Quản lý tài khoản



Hình 3.11. Biểu đồ luồng Quản lý tài khoản – Sửa, xóa người dùng



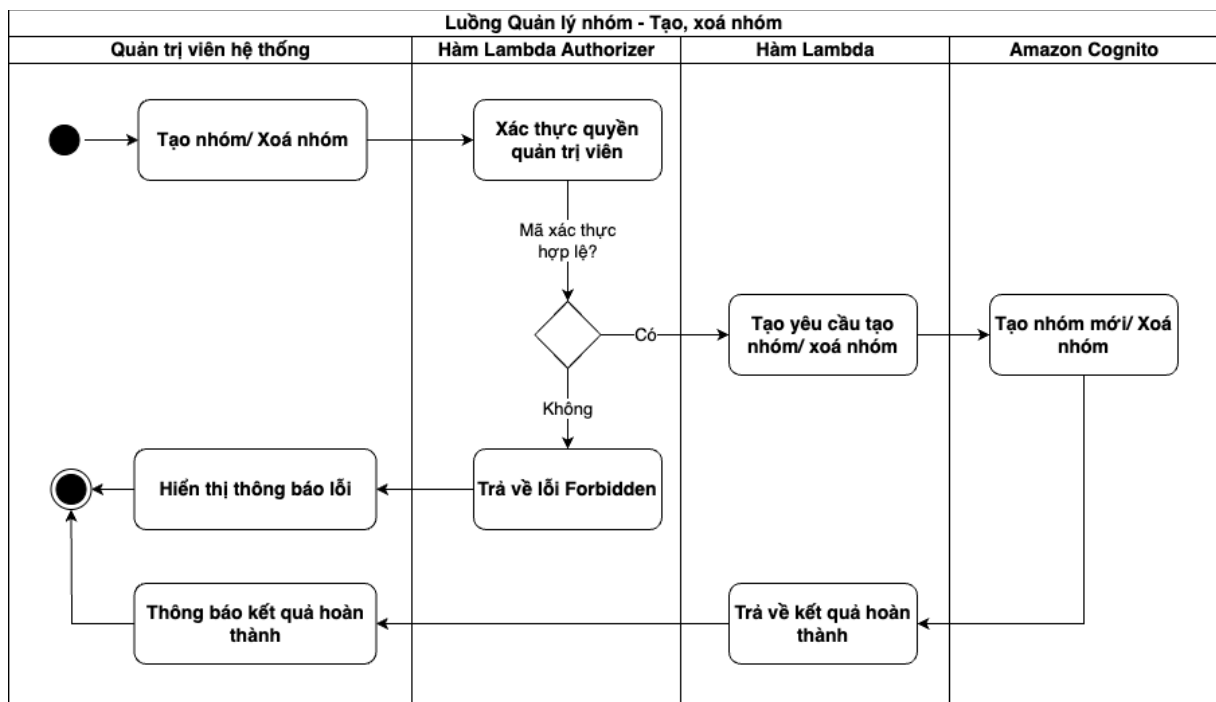
Hình 3.12. Biểu đồ luồng Quản lý tài khoản – Yêu cầu thay đổi mật khẩu

Biểu đồ Hình 3.11, 3.12 mô tả luồng Quản trị viên thực hiện tác vụ liên quan tới quản lý tài khoản người dùng. Quản trị viên có thể sửa đổi thông tin người dùng, xoá người dùng khỏi hệ thống hoặc tạo yêu cầu để người dùng thay đổi mật khẩu.

Để sửa đổi thông tin người dùng, xoá người dùng khỏi hệ thống, Quản trị viên thực hiện tìm kiếm người dùng, cập nhật thay đổi thông tin (nếu có) và xác nhận yêu cầu. Hàm Lambda Authorizer kiểm tra quyền thực hiện thay đổi và nếu mã xác thực quyền Quản trị viên hợp lệ, yêu cầu được chuyển tới hàm Lambda xử lý. Hàm này gọi các API phía Cognito để yêu cầu thay đổi thông tin của người dùng hoặc xoá người dùng khỏi hệ thống rồi trả về thông báo kết quả cho Quản trị viên.

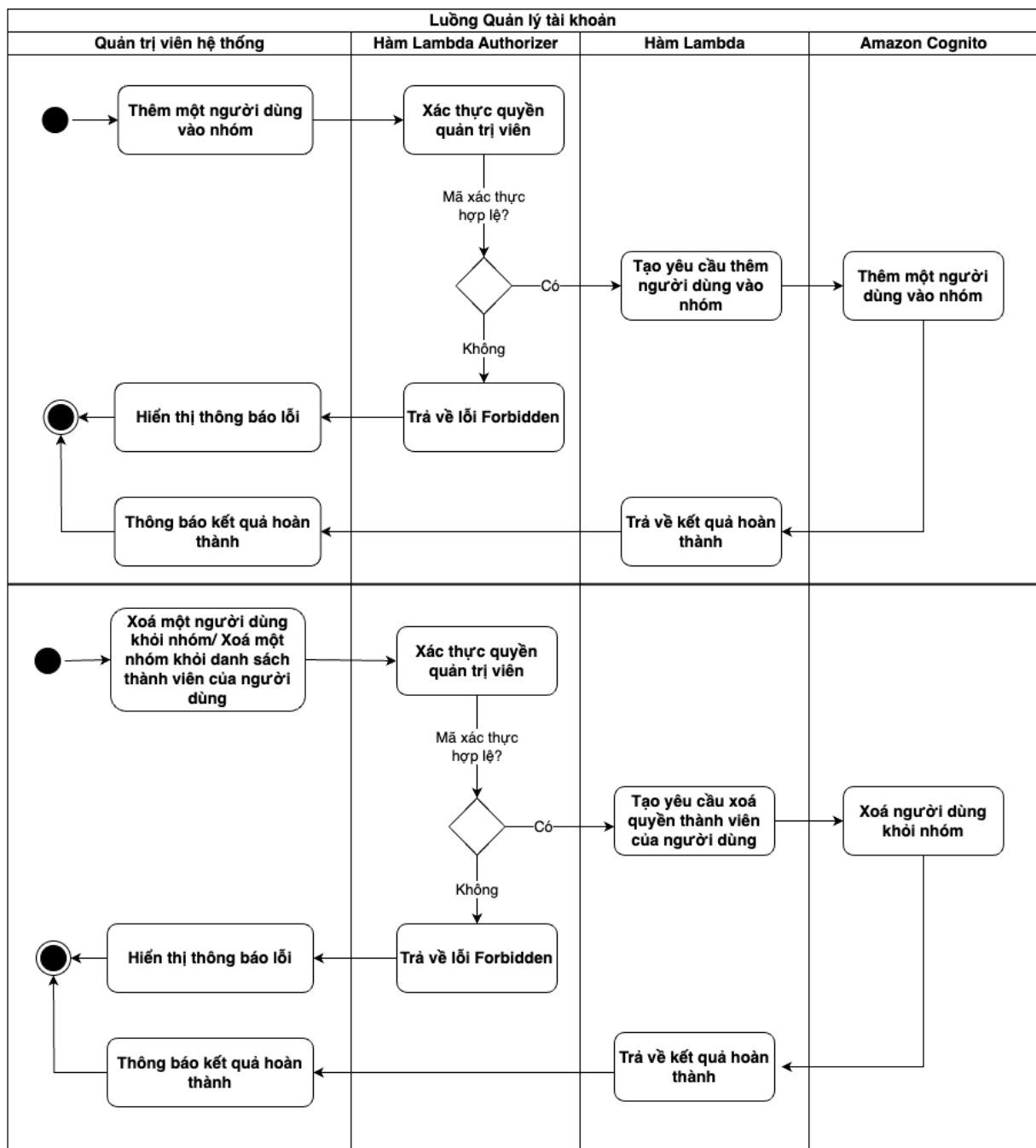
Để tạo yêu cầu cập nhật mật khẩu, sau khi nhận được yêu cầu mong muốn thay đổi mật khẩu từ phía cán bộ nhân viên, Quản trị viên tìm kiếm người dùng trên hệ thống và tạo yêu cầu thay đổi mật khẩu trên giao diện của hệ thống. Trước khi xử lý yêu cầu, hàm Authorizer kiểm tra tính hợp lệ và quyền thực hiện yêu cầu của mã xác thực. Nếu đảm bảo hợp lệ, yêu cầu được chuyển tiếp tới hàm Lambda xử lý. Hàm này thực hiện tạo yêu cầu thay đổi, lưu yêu cầu phiên thay đổi tới DynamoDB để xác thực yêu cầu khi người dùng thay đổi mật khẩu cũng như khoá phiên đăng nhập của người dùng buộc người dùng phải thay đổi mật khẩu mới có thể đăng nhập lại.

3.8. Thiết kế luồng Quản lý nhóm phân quyền



Hình 3.13. Biểu đồ luồng Quản lý nhóm phân quyền - Tạo, xoá nhóm

Ca sử dụng Quản lý nhóm phân quyền với chức năng Tạo, xoá nhóm được thể hiện ở Hình 3.13. Quản trị viên có thể quản lý quyền của các hệ thống nội bộ bằng các nhóm phân quyền. Quản trị viên có thể tạo một nhóm phân quyền mới hoặc xoá một nhóm phân quyền hiện có. Luồng thực hiện thao tác này bắt đầu bằng việc Quản trị viên thực hiện thao tác tạo, xoá nhóm phân quyền trên giao diện hệ thống SSO. Quản trị viên chỉ định tên nhóm phân quyền sẽ tạo/xoá (và các thông tin khác như độ ưu tiên, mô tả,... nếu thao tác là tạo nhóm phân quyền). Yêu cầu gửi tới API Gateway sẽ được tiền kiểm tra về quyền thực thi tại hàm Lambda Authorizer và nếu mã xác thực hợp lệ, API Gateway chuyển tiếp yêu cầu đến hàm Lambda xử lý tương ứng. Hàm Lambda tạo yêu cầu tạo nhóm/xoá nhóm tới Amazon Cognito để thực hiện tạo nhóm mới hoặc xoá nhóm hiện tại. Sau đó hàm Lambda này trả lại kết quả hoàn thành để hiển thị thông báo cho Quản trị viên.

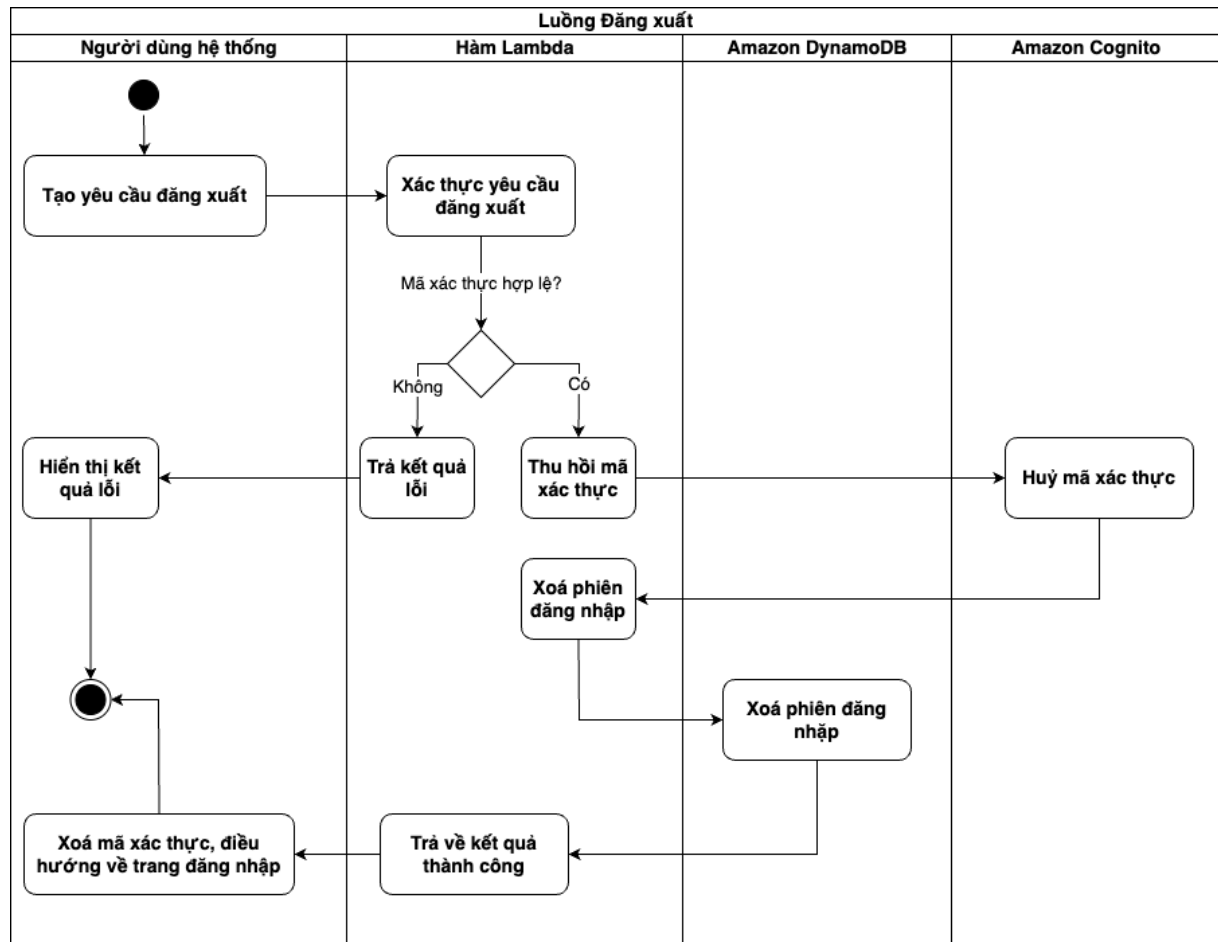


Hình 3.14. Biểu đồ luồng Thêm, xóa quyền thành viên nhóm của người dùng

Hình 3.14 cũng mô tả luồng Quản trị viên thêm/xóa quyền thành viên nhóm phân quyền của người dùng. Quản trị viên có thể lựa chọn một tài khoản trong danh sách tài khoản và thêm tài khoản này vào danh sách thành viên của nhóm. Khi này hàm Lambda Authorizer tiếp nhiệm yêu cầu sẽ kiểm tra quyền Quản trị viên của người dùng. Nếu hợp lệ, yêu cầu được xử lý tiếp tục ở một hàm Lambda, hàm này tạo yêu cầu tới Amazon Cognito để thêm người dùng được chỉ định vào nhóm và trả lại trạng thái thực hiện cho Quản trị viên. Tương tự với việc thêm người dùng, việc loại bỏ quyền truy cập của người dùng khỏi một nhóm cũng được thực hiện bằng việc Quản trị viên truy cập giao diện và

xác nhận loại bỏ một người dùng khỏi nhóm hoặc loại bỏ một nhóm khỏi danh sách nhóm phân quyền mà người dùng tham gia. Yêu cầu này được gửi tới API Gateway sau đó tiền xử lý về quyền tại hàm Lambda Authorizer. Yêu cầu này được xử lý tại hàm Lambda, thực hiện thay đổi về quyền của người dùng với nhóm phân quyền trên Amazon Cognito và thực hiện trả lại kết quả cho người dùng.

3.9. Thiết kế luồng Đăng xuất



Hình 3.15. Biểu đồ luồng Đăng xuất

Để đăng xuất khỏi hệ thống, người dùng có thể đăng xuất tại giao diện cá nhân của hệ thống SSO hoặc đăng nhập tại các hệ thống nội bộ như mô tả tại Hình 3.15. Khi người dùng tạo yêu cầu đăng xuất, hàm Lambda kiểm tra xem phiên đăng nhập của người dùng có hợp lệ không, nếu không sẽ trả về kết quả lỗi. Nếu phiên đăng nhập hợp lệ, hàm Lambda thực hiện yêu cầu thu hồi mã xác thực tại Cognito cũng như xoá thông tin về phiên đăng nhập được lưu tại DynamoDB. Hàm Lambda trả về kết quả đăng xuất thành công và hệ thống SSO hoặc các hệ thống nội bộ tiến hành xoá mã xác thực được lưu tại bộ nhớ cục bộ rồi điều hướng người dùng trở lại giao diện đăng nhập của hệ thống SSO.

3.10. Tổng kết chương

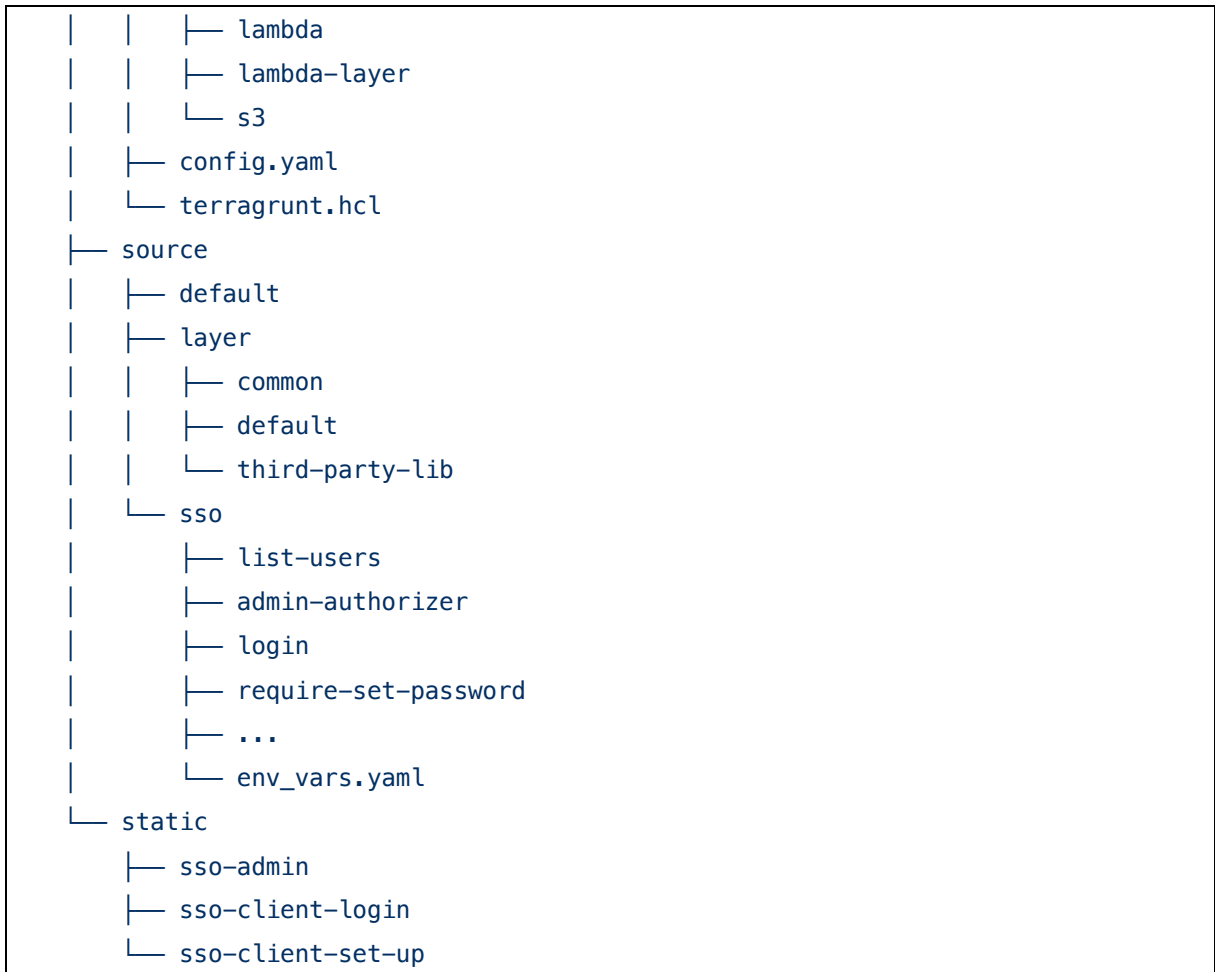
Chương 3 đã trình bày một cách chi tiết về việc phân tích và thiết kế hệ thống ở mức cao, mô tả luồng hoạt động tổng quan của hệ thống thông qua các dịch vụ đám mây của AWS cũng như các luồng của các ca sử dụng đã được mô tả ở Chương 2. Trong chương tiếp theo, KLTN sẽ tiếp tục giới thiệu về mô hình tổ chức mã nguồn của hệ thống, từ đó trình bày về cách thức cài đặt và triển khai hệ thống cũng như một số kết quả kiểm thử và thử nghiệm sử dụng hệ thống.

CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG, CÀI ĐẶT VÀ KIỂM THỬ

4.1. Xây dựng cấu trúc mã nguồn

Trước tiên, ta sẽ đi sâu vào cách thức tổ chức các thư mục mã nguồn của hệ thống. Như đã trình bày ở Chương 1, ta sẽ triển khai cấu trúc mã nguồn dưới dạng các mô-đun. Mô-đun trong Terraform hoạt động tương tự một hàm trong các ngôn ngữ lập trình, nó giúp ta ẩn đi các chi tiết mà người dùng không cần biết đến, khiến cho mã nguồn trở nên dễ đọc và dễ hiểu hơn, bên cạnh đó còn tăng khả năng tái sử dụng của mã nguồn [8]. Cấu trúc mã nguồn của hệ thống có thể được chia thành 3 phần chính: mã nguồn cho hạ tầng tài nguyên trên đám mây AWS (tương ứng với thư mục `infra` trong cây mã nguồn), mã nguồn cho phần xử lý mặt sau bằng AWS Lambda (tương ứng với thư mục `source` trong cây mã nguồn) và mã nguồn cho các trang web mặt trước (tương ứng với thư mục `static` trong cây mã nguồn) như được minh họa như Hình 4.1.





Hình 4.1. Mô tả cấu trúc cây mã nguồn của hệ thống

4.1.1. Mô tả thư mục static

Thư mục static bao gồm 3 thư mục con chứa mã nguồn cho các trang web mặt trước của hệ thống SSO được viết bằng ReactJS. Thư mục con sso-admin tập hợp mã nguồn cho giao diện trang web quản lý của Quản trị viên hệ thống. Thư mục con sso-client-login chứa mã nguồn cho giao diện mặt trước cho chức năng đăng nhập SSO của người dùng. Cuối cùng, thư mục con sso-client-set-up bao gồm mã nguồn xây dựng giao diện cho các chức năng liên quan đến thiết lập tài khoản phía người dùng như: đặt mật khẩu, đặt lại mật khẩu hay thiết lập phương thức xác thực đa yếu tố (nếu có).

4.1.2. Mô tả thư mục source

Thư mục source bao gồm các thư mục con chứa mã nguồn của các hàm Lambda xử lý các yêu cầu nghiệp vụ cũng như mã nguồn của các lớp Lambda. Cụ thể, các thư mục con trong thư mục layer tương ứng với các lớp Lambda. Các thư mục con trong thư mục sso tương ứng với một hàm Lambda. Như đã giới thiệu ở Chương 1, các lớp Lambda bao gồm mã nguồn của các thư viện bên thứ ba mà các hàm Lambda sẽ sử dụng (ví dụ như jwt, boto3,...) cũng như mã nguồn của một số hàm tùy biến dùng chung cho

các hàm Lambda để tránh tái định nghĩa lại trong từng hàm Lambda, tăng khả năng tái sử dụng của mã nguồn. Hình 4.2 và 4.3 minh họa hai hàm Python, trong đó Hình 4.2 là một hàm được đóng gói trong lớp Lambda, hỗ trợ tùy biến thư viện logging để lưu lại các sự kiện trong hệ thống.

<code>/layer/common/python/utils/logger.py</code>	
1	<code>import logging</code>
2	
3	<code>def get_logger(module: str = __name__) -> logging.Logger:</code>
4	<code> logger = logging.getLogger(module)</code>
5	<code> logger.setLevel(logging.DEBUG)</code>
6	<code> logger.propagate = False</code>
7	
8	<code> logger_handler = logging.StreamHandler()</code>
9	<code> logger_formatter = logging.Formatter(</code>
10	<code> '[(levelname)s] %(asctime)s - %(name)s, line %(lineno)d - %(message)s')</code>
11	<code>)</code>
12	<code> logger_handler.setFormatter(logger_formatter)</code>
13	<code> logger.addHandler(logger_handler)</code>
14	
15	<code> return logger</code>

Hình 4.2. Mẫu một hàm tùy biến được định nghĩa trong lớp Lambda

Hình 4.3 minh họa hàm Lambda thực hiện chức năng yêu cầu người dùng thiết lập mật khẩu. Hàm Lambda này nhận sự kiện yêu cầu từ API Gateway, lấy ra danh sách người dùng cần thiết lập mật khẩu, lưu phiên thiết lập mật khẩu và gửi email chứa đường dẫn tới giao diện thiết lập mật khẩu tới email của người dùng.

<code>/sso/require-set-password/require-set-password.py</code>	
1	<code>import hashlib</code>
2	<code>import json</code>
3	<code>from http import HTTPStatus</code>
4	
5	<code>from aws.dynamodb import DynamoDBService</code>
6	<code>from aws.ssm import SystemsManager</code>
7	<code>from utils.enums import ErrorCode</code>
8	<code>from utils.logger import get_logger</code>
9	<code>from utils.response import get_apigw_response, get_response_body</code>
10	<code>from utils.time import get_time_stamp_from_now</code>
11	
12	<code>logger = get_logger(__name__)</code>

```

13
14 ssm = SystemsManager()
15 cache_table = ssm.get_ssm_value("DYNAMO_CACHE_TABLE")
16 cf_domain = ssm.get_ssm_value("CF_DOMAIN")
17
18 dynamodb = DynamoDBService(table_name=cache_table)
19
20 set_password_domain = f"https://{cf_domain}/set-up/client"
21
22 def lambda_handler(event, context):
23     try:
24         logger.debug(f"Event captured: {json.dumps(event)}")
25
26         body = json.loads(event.get("body"))
27         user_list = body.get("user_list", [])
28
29         for new_user in user_list:
30             username = new_user.get("username")
31             email = new_user.get("email")
32
33             identifier = hashlib.sha256(
34                 f"{username}_{email}".encode("utf-8")).hexdigest()
35             set_password_url = f"{set_password_domain}?requestId={identifier}"
36
37             identifier_item = {
38                 "KeyID": identifier,
39                 "Digest": set_password_url,
40                 "Expiration": get_time_stamp_from_now(day_interval=1)
41             }
42             dynamodb.table.put_item(Item=identifier_item)
43             message = f"Reset password at: {set_password_url}"
44             send_email(email, message)
45             response_body = get_response_body(ErrorCode.SUCCESS.value, "Done")
46             return get_apigw_response(HTTPStatus.OK, response_body)
47         except Exception as err:
48             logger.error(f"Exception: {str(err)}")
49             response_body = get_response_body(ErrorCode.SYSTEM_ERROR.value,
50                 f"Exception: {str(err)}")
51             return get_apigw_response(HTTPStatus.INTERNAL_SERVER_ERROR,
52                 response_body)

```

Hình 4.3. Mẫu mã nguồn hàm Lambda yêu cầu đặt mật khẩu

Bên cạnh các thư mục chứa mã nguồn, thư mục source còn có một tệp `env_vars.yaml`. Tệp này chứa các định nghĩa về các biến môi trường cho mỗi hàm Lambda. Hình 4.4 mô tả một phần của tệp định nghĩa này, trong đó ta thiết lập hai biến môi trường là ENV và PROJECT cho hàm Lambda có tên `require-set-password` ở môi trường dev.

/sso/env_vars.yaml	
...	...
47	<code>require-set-password:</code>
48	<code> dev:</code>
49	<code> ENV: dev</code>
50	<code> PROJECT: sso</code>
51	<code> uat:</code>
52	<code> prod:</code>
...	...

Hình 4.4. Mẫu cấu hình biến môi trường cho một hàm Lambda

Toàn bộ mã nguồn của các hàm Lambda, lớp Lambda cùng các biến môi trường sẽ tự động được đóng gói dưới dạng tệp nén (.zip) và tải lên môi trường thực thi của Lambda khi có sự thay đổi trong mã nguồn nhờ vào cấu hình của Terraform được định nghĩa trong thư mục `infra` sẽ được trình bày chi tiết hơn ở phần tiếp theo.

4.1.3. Mô tả thư mục *infra*

Thư mục này chứa thông tin cấu hình hạ tầng các dịch vụ đám mây của hệ thống. Ta tổ chức mã nguồn IaC của hệ thống thành 4 thư mục, trong đó 3 thư mục dành cho 3 môi trường phát triển của hệ thống và thư mục `common` là thư mục định nghĩa các cấu hình chung cho 3 môi trường phát triển. Thư mục `common` bao gồm các tệp cấu hình Terraform định nghĩa cấu trúc chung của toàn bộ các tài nguyên cho các môi trường triển khai. Các thư mục tại mỗi môi trường chỉ gồm một tệp `Terragrunt` định nghĩa các cấu hình triển khai khác nhau trong mỗi môi trường (ví dụ kích thước bộ nhớ cấp phát, số lượng máy chủ,...). Việc triển khai cấu trúc thư mục như vậy giúp đảm bảo tính chất DRY của hệ thống, tức ta không cần phải lặp lại các cấu hình giống nhau ở từng môi trường phát triển mà chỉ cần tập trung vào các thành phần khác nhau, không những tăng khả năng tái sử dụng mã nguồn mà còn đảm bảo tránh xảy ra sai sót trong quá trình triển khai hạ tầng các dịch vụ đám mây giữa các môi trường.

Đối với mỗi thư mục dùng chung và dành cho mỗi môi trường phát triển, ta chia các tệp cấu hình thành 3 thư mục con ứng với các dịch vụ được triển khai trong 3 tầng

trong mô hình thiết kế của hệ thống (Hình 4.5). Việc phân chia như vậy giúp ta nhóm các dịch vụ có liên quan tới nhau để triển khai trong quá trình phát triển ứng dụng.

```

.
├── serverless
│   ├── archive
│   ├── policy
│   ├── template
│   ├── apigw.tf
│   ├── lambda.tf
│   ├── layer.tf
│   ├── locals.tf
│   └── variables.tf
├── static
│   ├── policy
│   ├── template
│   ├── cloudfront.tf
│   ├── locals.tf
│   ├── output.tf
│   ├── s3.tf
│   └── variables.tf
└── storage
    ├── policy
    ├── template
    ├── dynamodb.tf
    ├── locals.tf
    └── variables.tf

```

Hình 4.5. Cấu trúc thư mục common

Bên cạnh đó ta cũng định nghĩa trong tệp định nghĩa của Lambda một kiểu tài nguyên đặc biệt, kiểu tài nguyên này giúp ta tự đóng gói mã nguồn của các tệp Python trong thư mục source, triển khai các mã nguồn này lên môi trường thực thi của Lambda cũng như theo dõi sự thay đổi trong mã nguồn của các hàm Lambda (thông qua mã băm của các tệp đóng gói của mã nguồn) được định nghĩa trong thư mục source và đóng gói lại mã nguồn của các hàm Lambda có sự thay đổi một cách tự động.

common/serverless/lambda.tf

```

1  module "lambda_function" {
...      ...
9      source          = "github.com/datngxtiens/terraform-module.git//lambda"
10     for_each        = local.lambda_functions

```

```

...
24 filename          = "archive/${each.value.project}/${each.key}.zip"
25 source_code_hash = (lookup(each.value, "custom_source", false) ?
26 filebase64sha256("archive/${each.value.project}/${each.key}.zip") : null)
27 timeout          = var.lambda_timeout
...
57 apigw_id        = module.api_gateway.id
58 }
59
60 data "archive_file" "lambda_zip" {
61   for_each = local.lambda_functions
62   type     = "zip"
63   source_dir = (lookup(each.value, "custom_source", false) ?
64     "${var.source_path}/source/${each.value.project}/${each.key}" :
65     "${var.source_path}/source/default"
66   )
67   output_path = "archive/${each.value.project}/${each.key}.zip"
68 }

```

Hình 4.6. Một phần mã nguồn định nghĩa hàm Lambda

dev/serverless/terragrunt.hcl	
1	include "root" {
2	path = find_in_parent_folders()
3	}
4	include "vars" {
5	path = "../locals.hcl"
6	merge_strategy = "deep"
7	}
8	terraform {
9	source = "../../common/serverless"
10	}
11	
12	locals {
13	environment = "dev"
14	}
15	
16	inputs = {
17	lambda_memory_size = 128
18	lambda_timeout = 5
19	}

Hình 4.7. Tập định nghĩa Terragrunt cho môi trường dev

Tệp định nghĩa ở Hình 4.6 được thiết kế dưới dạng mô-đun để có thể tái sử dụng giữa các môi trường thực thi. Từ đó, ta có thể thiết lập các tệp định nghĩa tương tự như Hình 4.7 cho mỗi môi trường thực thi, truyền vào các thông số cấu hình đặc trưng của mỗi môi trường thực thi.

4.2. Các yêu cầu về cài đặt

Để có thể cài đặt và triển khai hệ thống, trước hết ta cần cài đặt một số công cụ hỗ trợ phát triển như:

- Python (≥ 3.8)
- Terraform ($\geq v1.4.2$)
- Terragrunt ($\geq v0.45$)
- AWS CLI ($\geq 2.10.1$)

4.3. Triển khai hệ thống trên môi trường đám mây

Trước tiên, để triển khai hệ thống trên môi trường đám mây, ta cần thực hiện một số cấu hình liên quan tới thông tin đăng nhập vào các tài khoản trên môi trường đám mây của AWS thông qua AWS CLI, điều chỉnh các thông tin cấu hình này trong tệp `config.yaml` trong thư mục `infra`.

Sau khi thực hiện cấu hình các thông tin về tài khoản AWS, trên Terminal ta di chuyển tới các thư mục chỉ môi trường muốn thực thi (ví dụ môi trường `dev`, `uat`, `prod`) trong thư mục `infra`. Ta tiếp tục di chuyển vào lần lượt từng thư mục trong thư mục môi trường và lần lượt thực hiện các câu lệnh sau:

Chạy câu lệnh `terragrunt init` để khởi tạo môi trường thực thi. Terraform sẽ khởi tạo và tải về các thư viện, mô-đun cần sử dụng cũng như lưu lại trạng thái cấu hình của hệ thống tới S3. Kết quả của câu lệnh này tương tự như đầu ra mô tả trong Hình 4.8.

```
~/infra/dev/serverless terragrunt init

Initializing the backend...

Successfully configured the backend "s3"! Terraform will automatically
use this backend unless the backend configuration changes.
Initializing modules...
Downloading git::https://github.com/datngxtiens/terraform-module.git for
api_gateway...
- api_gateway in .terraform/modules/api_gateway/apigw
Downloading ...
...
```

Initializing provider plugins...

- Finding hashicorp/aws versions matching ">= 2.0.0, >= 3.0.0"...
- Finding latest version of hashicorp/archive...
- Installing hashicorp/aws v4.62.0...
- Installed hashicorp/aws v4.62.0 (signed by HashiCorp)
- Installing hashicorp/archive v2.3.0...
- Installed hashicorp/archive v2.3.0 (signed by HashiCorp)

Terraform has created a lock file **.terraform.lock.hcl** to record the provider selections it made above. Include this file in your version control repository so that Terraform can guarantee to make the same selections by default when you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

Hình 4.8. Khởi tạo môi trường thực thi

Tiếp đó, ta có thể chạy câu lệnh `terraform plan` để xem các thay đổi sẽ xảy ra với hạ tầng hệ thống. Ta cũng có thể trực tiếp chạy câu lệnh `terraform apply` để xem các thay đổi cũng như xác nhận các thay đổi này (Hình 4.9). Sau khi chạy câu lệnh `terraform apply`, Terminal hiển thị các thay đổi sẽ xảy ra đối với các tài nguyên trên môi trường đám mây của AWS. Để xác nhận các thay đổi, ta gõ `yes`. Như đã trình bày trước đó, ta đã cấu hình để Terraform tự động đóng gói các thư mục mã nguồn dưới dạng các tệp zip và áp dụng nó vào các hàm Lambda tương ứng. Sau khi gõ `yes`, Terraform sẽ tiến hành khởi tạo các tài nguyên theo các tệp cấu hình có trong thư mục `infra` và ta chỉ cần đợi cho tới khi quá trình này kết thúc.

```
~/infra/dev/serverless terraform apply
```

```
Acquiring state lock. This may take a few moments...
```

```
data.archive_file.layer_zip["third-party-lib"]: Reading...
```

```
data.archive_file.layer_zip["common"]: Reading...
```

```

data.archive_file.layer_zip["common"]: Read complete after 0s
[id=3835fe5551088915ca050500014d7bd20dabe17f]
data.archive_file.layer_zip["third-party-lib"]: Read complete after 1s
[id=c349cde25ad517da7205b167e6b811a5a638d6d9]
data.aws_caller_identity.current: Reading...
data.archive_file.lambda_zip["login"]: Reading...
data.archive_file.lambda_zip["verify"]: Reading...
...
data.archive_file.lambda_zip["login"]: Read complete after 0s
[id=9154939e7f06fdbd2985c7df97b11485a790682e]
data.archive_file.lambda_zip["verify"]: Read complete after 0s
[id=df603f4f8e36e0145f02034afac504ec61392cc1]
...
Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
  + create
  <= read (data resources)

Terraform will perform the following actions:

# data.aws_lambda_layer_version.lambda_layer["common"] will be read during
apply
# (depends on a resource or a module with changes pending)
<= data "aws_lambda_layer_version" "lambda_layer" {
  + arn                        = (known after apply)
  + compatible_architectures  = (known after apply)
  + compatible_runtimes       = (known after apply)
  ...
  + layer_arn                 = (known after apply)
  + layer_name                 = "dev-uet-sso-layer-common"
  ...
  + source_code_hash          = (known after apply)
  + source_code_size           = (known after apply)
  + version                    = (known after apply)
}
...
# module.lambda_function["admin-authorizer"].aws_lambda_function.this will be
created
+ resource "aws_lambda_function" "this" {
  + architectures              = [
    + "x86_64",
  ]

```



```

+ arn = (known after apply)
+ description = "dev-uet-sso-lambda-admin-authorizer"
+ filename = "archive/sso/admin-authorizer.zip"
+ function_name = "dev-uet-sso-lambda-admin-authorizer"
+ handler = "admin-authorizer.lambda_handler"
...
+ layers = (known after apply)
+ memory_size = 128
+ package_type = "Zip"
...
+ role = (known after apply)
+ runtime = "python3.8"
...
+ source_code_hash = "wk9QjWzpySTv4iNsCdYj/mSC+YZe/Nnm5SPuc+ihTK8="
+ source_code_size = (known after apply)
+ tags = {
  + "Env" = "dev"
  + "Name" = "dev-uet-sso-lambda-admin-authorizer"
  + "cost-allocation:Project" = "sso"
  + "operations:Creator" = "datngxtiens"
}
+ tags_all = {
  + "Env" = "dev"
  + "Name" = "dev-uet-sso-lambda-admin-authorizer"
  + "cost-allocation:Project" = "sso"
  + "operations:Creator" = "datngxtiens"
}
+ timeout = 10
+ version = (known after apply)
}
...

```

Plan: 39 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value:

Hình 4.9. Áp dụng tệp cấu hình và khởi tạo tài nguyên

Sau khi hoàn thành việc áp dụng cấu hình cho tất cả các thư mục, ta đã có đầy đủ các tài nguyên cần thiết cũng như mã nguồn đã được triển khai sẵn sàng để đưa hệ thống vào hoạt động.

4.4. Đánh giá hiệu quả và kiểm thử hệ thống

4.4.1. Hiệu quả của hàm Lambda

Một cách tổng quát, việc sử dụng hàm Lambda cho thấy tốc độ xử lý và phản hồi đảm bảo được về yêu cầu về độ trễ mà bài toán đặt ra đã đề cập ở Chương 2. Như đã đề cập ở Chương 1, hàm Lambda trước khi có thể thực thi mã nguồn cần phải khởi tạo môi trường thực thi, gọi là “cold start”. Vì lẽ đó, đối với lần đầu tiên hàm Lambda được gọi thực thi, thời gian để hàm Lambda thực thi có độ trễ lớn hơn đáng kể so với thời gian khi hàm Lambda vào giai đoạn hoạt động ổn định, tức khi môi trường thực thi đã được khởi tạo ổn định, gọi là “warm start”. Khi này môi trường thực thi của Lambda chưa thực sự bị chấm dứt mà vẫn tiếp tục duy trì một thời gian để có thể tiếp tục xử lý các yêu cầu tiếp theo.

Chính đặc điểm này của hàm Lambda giúp AWS Lambda đảm bảo được sự cân bằng giữa yếu tố chi phí, giúp làm giảm thời gian thực thi của hàm Lambda nhưng vẫn giữ được đặc điểm của dịch vụ điện toán phi máy chủ đó là không cần phải duy trì trạng thái hoạt động của máy chủ liên tục. Điều này đặc biệt phù hợp với đặc điểm của ca sử dụng Đăng nhập mà bài toán đã đề cập. Thật vậy, đối với bài toán xác thực người dùng cho các doanh nghiệp, thông thường người dùng sẽ truy cập vào hệ thống cùng một lúc vào đầu ngày để thực hiện đăng nhập và sử dụng các dịch vụ nội bộ của hệ thống. Hàm Lambda không những có khả năng tự động mở rộng số lượng môi trường thực thi đồng thời để có thể đáp ứng số lượng yêu cầu tại một thời điểm nhanh chóng (lên tới 1000 hàm thực thi đồng thời, hay nói cách khác xử lý được 1000 yêu cầu tại một thời điểm) mà còn có khả năng xử lý nhanh, duy trì được môi trường thực thi liên tục để giảm độ trễ thực thi. Trong xuyên suốt ngày, khi số lượng yêu cầu thực thi giảm xuống, hàm Lambda có thể dừng các môi trường thực thi, từ đó giúp làm giảm chi phí sử dụng các hàm Lambda cũng như tránh gây lãng phí các nguồn tài nguyên công nghệ thông tin.

Để đánh giá về khả năng mở rộng và xử lý đồng thời của AWS Lambda, ta sẽ sử dụng công cụ kiểm thử hiệu năng Artillery²⁵. Artillery là công cụ mã nguồn mở, hỗ trợ khả năng kiểm thử độ chịu tải của hệ thống. Việc kiểm thử khả năng chịu tải của hệ thống đóng vai trò quan trọng, nhất là trong bối cảnh bài toán đặt ra yêu cầu khả năng

²⁵ <https://www.artillery.io/docs/>

chịu tải lớn khi số lượng nhân viên cùng lúc truy cập vào hệ thống. Để có thể sử dụng Artillery, trước tiên ta cần cài đặt và cấu hình các yêu cầu của bài kiểm tra chịu tải. Hình 4.10 và 4.11 mô tả tệp cấu hình và hướng dẫn cài đặt cũng như khởi chạy Artillery. Ở tệp cấu hình, ta thiết lập các giá trị bao gồm mục tiêu kiểm thử (target, url) cũng như các thông số cấu hình về mức độ chịu tải.

Cụ thể, để kiểm tra khả năng chịu tải của hệ thống, ta sẽ thực hiện bài kiểm tra chịu tải cho hoạt động đăng nhập, chia làm ba giai đoạn. Giai đoạn 1 mô tả khi cán bộ nhân viên của doanh nghiệp bắt đầu tới nơi làm việc và thực hiện hoạt động đăng nhập. Ta giả lập giai đoạn này kéo dài 1 phút (được định nghĩa bởi trường duration), với 5 yêu cầu được gửi đi mỗi giây (được định nghĩa bởi trường arrivalRate). Giai đoạn 2 giả lập phần kiểm thử cao tải, trong đó số lượng yêu cầu mỗi giây sẽ được tăng dần từ 5 yêu cầu (arrivalRate) tới 20 yêu cầu (rampTo) trong khoảng thời gian 2 phút. Ta giả lập giai đoạn này đồng bộ với khi phần lớn nhân viên tới công sở và thực hiện hoạt động đăng nhập vào hệ thống. Giai đoạn 3 được thực hiện trong vòng 10 phút, với 50 yêu cầu được thực hiện trong một giây nhằm mục đích kiểm thử khả năng xử lý số lượng yêu cầu lớn trong khoảng thời gian liên tục để đánh giá số lượng yêu cầu tối đa mà hệ thống có thể xử lý.

config.yaml

```
config:
  target: 'https://d2xgbj4u54ra14.cloudfront.net'
  phases:
    - duration: 60
      arrivalRate: 5
      name: WarmUp
    - pause: 10
    - duration: 120
      arrivalRate: 5
      rampTo: 20
      name: Ramp up load
    - pause: 30
    - duration: 600
      arrivalRate: 50
      name: Sustained load
  processor: './load_user.js'
  ensure:
    thresholds:
      - "http.response_time.p99": 500
```

```

scenarios:
  - flow:
    - function: "generate_user_cred"
    - post:
      url: "/lambda-api/login"
      json:
        username: "{{ username }}"
        password: "{{ password }}"
    - log: "Authenticated {{ username }} user."

```

Hình 4.10. Cấu hình kiểm tra chịu tải

```

~/infra/dev/serverless npm install -g artillery
~/infra/dev/serverless artillery run config.yaml

```

Hình 4.11. Cài đặt và chạy Artillery

```

All VUs finished. Total time: 13 minutes, 40 seconds

-----
Summary report @ 20:52:06(+0700)
-----

http.codes.200: ..... 11485
http.codes.400: ..... 19675
http.codes.500: ..... 1
http.codes.502: ..... 639
http.request_rate: ..... 42/sec
http.requests: ..... 31800
http.response_time:
  min: ..... 245
  max: ..... 4640
  median: ..... 584.2
  p95: ..... 1755
  p99: ..... 3262.4
http.responses: ..... 31800
vusers.completed: ..... 31800
vusers.created: ..... 31800
vusers.created_by_name.0: ..... 31800
vusers.failed: ..... 0
vusers.session_length:
  min: ..... 316.6
  max: ..... 5028.1
  median: ..... 671.9

```

p95:		1939.5
p99:		3328.3

Hình 4.12. Kết quả kiểm thử chịu tải

Log streams (50)		Refresh Delete Create log stream Search all log streams
Filter log streams or try prefix search		☐ Exact match ☐ Show expired Info < 1 > Settings
☐	Log stream	Last event time
☐	2023/05/20/[\$LATEST]8130e598ead34a588a544933c602cfff0	2023-05-20 20:48:28 (UTC+07:00)
☐	2023/05/20/[\$LATEST]1ba165412edd4dd382c46e9b029e9666	2023-05-20 20:48:28 (UTC+07:00)
☐	2023/05/20/[\$LATEST]0c8e27a733b747eea7f4ed6cd9da9a85	2023-05-20 20:48:28 (UTC+07:00)
☐	2023/05/20/[\$LATEST]36a1cb831bf94317b337d8a3159f39ae	2023-05-20 20:48:28 (UTC+07:00)
☐	2023/05/20/[\$LATEST]61d3cde8fb6e4b03b08f8b10e0133832	2023-05-20 20:48:08 (UTC+07:00)
☐	2023/05/20/[\$LATEST]44e4a191451748a08257b3db5eaf1891	2023-05-20 20:48:04 (UTC+07:00)
☐	2023/05/20/[\$LATEST]9585d188e2ca40beb40d77140f84c910	2023-05-20 20:48:04 (UTC+07:00)
☐	2023/05/20/[\$LATEST]d676a32431a94c3aab4d30d043f69a40	2023-05-20 20:48:04 (UTC+07:00)
☐	2023/05/20/[\$LATEST]b0cd0a104ba24be9bf2ca4121b10ce26	2023-05-20 20:47:49 (UTC+07:00)
☐	2023/05/20/[\$LATEST]cb6f2392061041c9be5811061046611b	2023-05-20 20:47:49 (UTC+07:00)
☐	2023/05/20/[\$LATEST]2f02fe0deb8e42878483ce7a40da4654	2023-05-20 20:47:49 (UTC+07:00)
☐	2023/05/20/[\$LATEST]4586d4b041df47b185c83e29d64a1fd3	2023-05-20 20:47:49 (UTC+07:00)
☐	2023/05/20/[\$LATEST]f5124267050747a0b4a396bc8728adbe	2023-05-20 20:47:49 (UTC+07:00)
☐	2023/05/20/[\$LATEST]bea2c0e2792843cba37227780e244af	2023-05-20 20:47:49 (UTC+07:00)

Hình 4.13. Hàm Lambda mở rộng số lượng môi trường thực thi

Hình 4.12 mô tả kết quả cuối cùng của bài kiểm thử chịu tải của hệ thống. Ta có thể đánh giá với khoảng thời gian gần 14 phút thực hiện kiểm thử, bài kiểm thử đã thực hiện được tổng cộng 31.800 yêu cầu, trung bình 42 yêu cầu được thực hiện mỗi giây. Trong số các yêu cầu được gửi tới hệ thống trong bài kiểm thử, có 31.160 yêu cầu được hệ thống thực xử lý thành công (trong đó có 11.485 yêu cầu được trả về mã lỗi 200, ứng với việc xác thực và cấp mã xác thực cho người dùng thành công; 19.675 yêu cầu trả về mã lỗi 400, ứng với việc hệ thống chặn người dùng đăng nhập và không trả về mã xác thực do người dùng đã thực hiện đăng nhập với mật khẩu sai hoặc đăng nhập quá số lần quy định, theo mô tả trong yêu cầu chức năng của bài toán). Hàm Lambda xử lý yêu cầu xảy ra lỗi 1 lần duy nhất (ứng với mã lỗi 500) và có 639 yêu cầu không được thực thi do quá tải lượng của API Gateway (ứng với mã lỗi 502).

Bên cạnh đó, ta cũng có thể quan sát một số các thông số kết quả khác bao gồm thời gian phản hồi nhỏ nhất (245 mili giây), thời gian phản hồi lớn nhất (4.640 mili giây) cũng như thời gian phản hồi trung vị của 31.800 yêu cầu kể trên (584,2 mili giây). Từ Hình 4.13, ta cũng có thể nhận thấy khả năng mở rộng số lượng môi trường thực thi

nhANH chóng khi số lượng yêu cầu tăng cao. Trong vòng gần 14 phút kiểm thử, Lambda đã mở rộng tới 50 môi trường thực thi đồng thời, trong đó có 6 môi trường thực thi được cấp phát ngay trong giây đầu tiên có số lượng yêu cầu được gửi tới. Với những thông số trên, ta có thể đảm bảo về khả năng chịu tải cũng như khả năng mở rộng để xử lý các yêu cầu của hệ thống khi số lượng yêu cầu được gửi tới tăng cao, ngay cả với những doanh nghiệp có số lượng nhân viên lớn.

Về mặt chi phí, so sánh với việc sử dụng các máy chủ thông thường, tham khảo từ trang tính giá của AWS²⁶, ta có thể thấy mức giá của việc sử dụng Lambda mang lại sự tiết kiệm khá lớn so với việc sử dụng máy chủ thông thường, ví dụ như với máy ảo EC2 như mô tả ở Bảng 4.1.

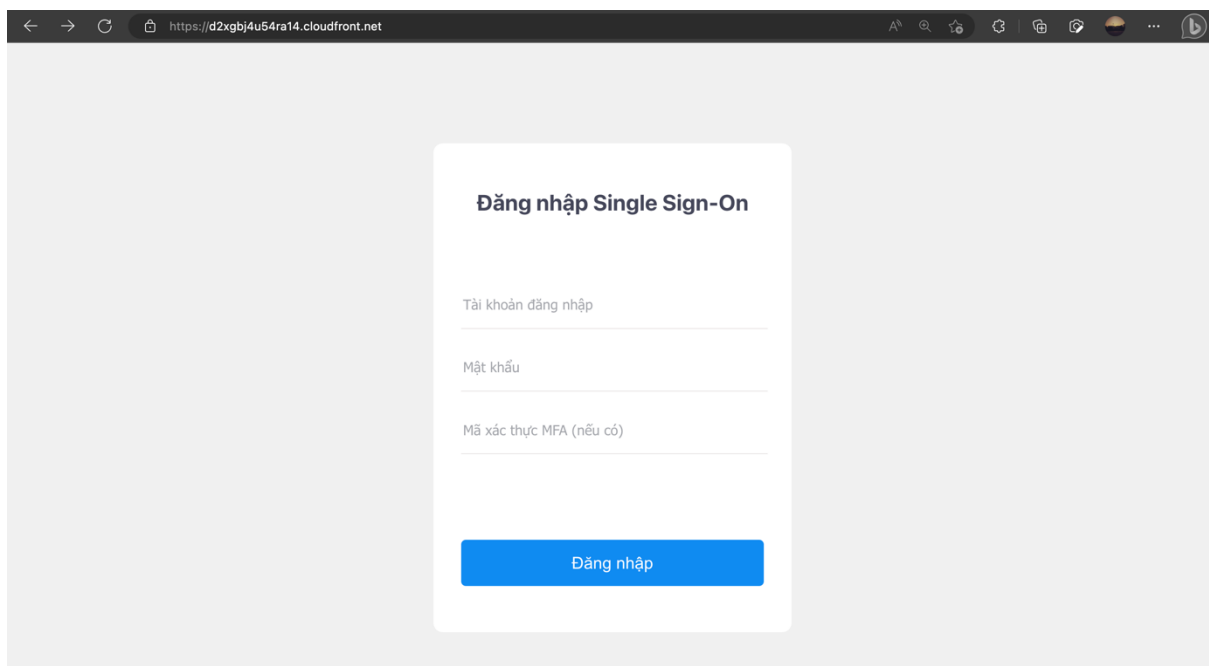
	Lambda Function	EC2
Thông số	- Kiến trúc: x86 - Số lượng yêu cầu/ngày: 100.000 - Bộ nhớ phân bổ: 512MB - Bộ nhớ tạm thời: 512MB - Thời gian thực thi trung bình: 5000 mili giây	- Cơ chế tính giá: On-Demand, không trả trước - Loại: t3.xlarge - vCPU: 4 - Bộ nhớ: 16GiB - Số lượng máy ảo tối thiểu: 2 - Số lượng máy ảo tối đa: 3
Mức giá	4,19 USD/tháng	204,26 USD/tháng

Bảng 4.1. So sánh mức giá Lambda và EC2

4.4.2. Kiểm thử chức năng Đăng nhập SSO

Vì các lý do chủ quan liên quan đến kinh nghiệm trong hoạt động kiểm thử của cá nhân, các ca kiểm thử ở mục này được thực hiện một cách thủ công, mà không sử dụng bất cứ một công cụ kiểm thử nào. Với vai trò là người dùng của hệ thống, ta sẽ tập trung vào kiểm thử các chức năng của hệ thống Đăng nhập SSO và cách ca sử dụng liên quan đảm bảo các yêu cầu chức năng của hệ thống đã được đề cập ở Chương 2.

²⁶ <https://calculator.aws/>



Hình 4.14. Giao diện Đăng nhập SSO

Sau khi truy cập vào các hệ thống nội bộ hoặc hệ thống Đăng nhập SSO như Hình 4.14, ta tiến hành thực hiện các ca kiểm thử được trình bày ở Bảng 4.2. Bảng 4.3 trình bày các kết quả đạt được với các ca kiểm thử đã trình bày ở Bảng 4.2.

STT	Yêu cầu chức năng	Các bước kiểm thử
1.1	Đăng nhập SSO	1. Truy cập hệ thống Quản trị nhân lực. 2. Nhập chính xác thông tin đăng nhập tại giao diện Đăng nhập. 3. Truy cập hệ thống Quản lý công việc.
1.2	Người dùng thiết lập mật khẩu	1. Truy cập giao diện Thiết lập mật khẩu. 2. Thiết lập mật khẩu mới. 3. Đăng nhập vào hệ thống bằng mật khẩu vừa mới tạo.
1.3	Nhân viên truy cập hệ thống Quản trị viên	1. Truy cập giao diện quản lý của Quản trị viên trong phiên đăng nhập với tài khoản không được cấp quyền.
1.4	Đăng xuất khỏi hệ thống	1. Truy cập hệ thống Quản trị nhân lực. 2. Nhấn Đăng xuất khỏi hệ thống. 3. Truy cập hệ thống Quản lý công việc.

1.5	Đăng nhập sai quá 5 lần.	1. Truy cập giao diện Đăng nhập SSO. 2. Đăng nhập sai tài khoản/mật khẩu quá 5 lần. 3. Đăng nhập lại với tài khoản và mật khẩu đúng. 4. Đợi 10 phút và đăng nhập lại với tài khoản và mật khẩu đúng.
-----	--------------------------	---

Bảng 4.2. Danh sách các ca kiểm thử Đăng nhập SSO

STT	Kết quả mong muốn	Trạng thái
1.1	1. Điều hướng sang giao diện Đăng nhập khi truy cập hệ thống Quản trị nhân lực. 2. Điều hướng trở lại hệ thống Quản trị nhân lực khi đăng nhập thành công. 3. Truy cập hệ thống Quản lý công việc mà không cần đăng nhập lại.	Đạt
1.2	1. Mật khẩu mới được thiết lập. 2. Đăng nhập thành công với mật khẩu mới tạo.	Đạt
1.3	1. Hệ thống hiển thị người dùng không được quyền truy cập.	Đạt
1.4	1. Người dùng được điều hướng trở lại trang Đăng nhập SSO. 2. Người dùng không truy cập được vào hệ thống Quản lý công việc và được điều hướng trở lại trang đăng nhập SSO.	Đạt
1.5	1. Hiển thị sai tài khoản/mật khẩu với 5 lần đăng nhập sai đầu tiên. 2. Hiển thị khoá tài khoản, yêu cầu đăng nhập lại sau 10 phút. 3. Đăng nhập thành công sau 10 phút.	Đạt

Bảng 4.3. Kết quả các ca kiểm thử Đăng nhập SSO

4.4.3. Kiểm thử chức năng phía Quản trị viên

Bên cạnh các ca kiểm thử phía người dùng là nhân viên, ta cũng sẽ kiểm thử các ca sử dụng với các ca kiểm thử cho các chức năng phía Quản trị viên. Bảng 4.4 và 4.5 mô tả các ca kiểm thử cho các chức năng phía Quản trị viên cũng như kết quả đạt được.

STT	Yêu cầu chức năng	Các bước kiểm thử
2.1	Quản trị viên thêm tài khoản mới	1. Nhập mã xác thực tài khoản Quản trị viên vào mục Headers của yêu cầu. 2. Nhập các thông tin của người dùng.
2.2	Quản trị viên thêm tài khoản bằng tệp	1. Nhập mã xác thực tài khoản Quản trị viên vào mục Headers của yêu cầu. 2. Tải lên tệp chứa thông tin cá nhân của người dùng.
2.3	Quản trị viên thêm nhóm nghiệp vụ	1. Nhập mã xác thực tài khoản Quản trị viên vào mục Headers của yêu cầu. 2. Nhập các thông tin về nhóm phân quyền.
2.4	Quản trị viên thêm người dùng vào nhóm nghiệp vụ	1. Nhập mã xác thực tài khoản Quản trị viên vào mục Headers của yêu cầu. 2. Nhập các thông tin về người dùng cần thêm và nhóm phân quyền.

Bảng 4.4. Danh sách ca kiểm thử chức năng phía Quản trị viên

STT	Kết quả mong muốn	Trạng thái
2.1	1. Người dùng được thêm thành công vào Cognito. 2. Email chứa đường dẫn thiết lập mật khẩu được gửi thành công tới người dùng.	Đạt yêu cầu 1, yêu cầu 2 chưa gửi được email do chưa đăng ký được email trên dịch vụ Amazon SES
2.2	1. Tệp người dùng được thêm vào Cognito. 2. Email chứa đường dẫn thiết lập mật khẩu được gửi thành công tới người dùng.	Đạt yêu cầu 1, yêu cầu 2 chưa gửi được email do chưa đăng ký được email trên dịch vụ Amazon SES
2.3	1. Nhóm mới được thêm trên Cognito.	Đạt
2.4	1. Người dùng được thêm vào danh sách thành viên của nhóm.	Đạt

Bảng 4.5. Kết quả các ca kiểm thử chức năng phía Quản trị viên

4.5. Tổng kết chương

Từ những lý thuyết đã được trình bày ở Chương 1, phát triển từ bài toán đặt ra ở Chương 2 cũng như hiện thực hoá những thiết kế đã được đề cập ở Chương 3, chương này đã giới thiệu về mô hình triển khai mã nguồn của hệ thống, hướng dẫn cài đặt để hệ thống vận hành trên môi trường đám mây của AWS và đã kiểm thử hoạt động của hệ thống ở một số trường hợp cơ bản cũng như chứng minh được tính hiệu quả đáp ứng những yêu cầu phi chức năng đã đặt ra của việc sử dụng dịch vụ điện toán phi máy chủ trong bài toán xác thực SSO.

KẾT LUẬN

Với sự phát triển của khoa học công nghệ, đặc biệt là các dịch vụ điện toán đám mây, nhu cầu sử dụng các dịch vụ điện toán đám mây đang ngày càng trở nên phổ biến trong các doanh nghiệp. Xuất phát từ bài toán xây dựng hệ thống SSO cho hoạt động xác thực nội bộ của doanh nghiệp, kết hợp với các dịch vụ phi máy chủ trên nền tảng điện toán đám mây của AWS, đề tài “**Xây dựng Hệ thống SSO phi máy chủ**” đã lên ý tưởng và xây dựng một giải pháp cho doanh nghiệp xác thực người dùng một lần cho toàn bộ các hệ thống vệ tinh nội bộ của doanh nghiệp. Hệ thống được xây dựng đảm bảo được yêu cầu cốt lõi đó là người dùng chỉ cần đăng nhập một lần trên hệ thống SSO và có thể sử dụng mã xác thực được cấp để truy cập và sử dụng các dịch vụ và sản phẩm phần mềm nội bộ của doanh nghiệp. Hệ thống SSO cũng đã tích hợp công cụ quản lý người dùng và phân quyền cho Quản trị viên của hệ thống, giúp làm thuận tiện hơn quá trình vận hành của các Quản trị viên của hệ thống. Bên cạnh đảm bảo được các yêu cầu về chức năng, hệ thống được xây dựng và triển khai trong KLTN này cũng đã đảm bảo được các yêu cầu phi chức năng khác mà bài toán đặt ra. Với việc triển khai hệ thống trên nền tảng của các dịch vụ phi máy chủ, doanh nghiệp không cần phải sở hữu hay quản lý hạ tầng phần cứng nhưng lại dễ đến đến lãng phí các tài nguyên không được sử dụng tới trong thời gian rảnh rỗi và không đủ đáp ứng yêu cầu khi lượng truy cập gia tăng đột biến.

Từ những nghiên cứu và tìm hiểu có được qua KLTN này, tôi nhận thấy bản thân đã có được thêm nhiều trải nghiệm mới về một lĩnh vực đang có tiềm năng phát triển rất lớn tại Việt Nam là điện toán đám mây. Bài toán đặt ra trong KLTN này không những đòi hỏi yêu cầu về việc phân tích, đặc tả yêu cầu và xây dựng phần mềm mà còn giúp tôi có thêm nhiều kiến thức mới về thiết kế hệ thống, xây dựng kiến trúc phần mềm và quản lý tài nguyên điện toán đám mây hiệu quả. Tôi cũng mong rằng đây cũng có thể là một nguồn tài liệu tham khảo cho các bạn sinh viên có mong muốn tìm hiểu về các dịch vụ điện toán đám mây của AWS cũng như về việc phát triển các hệ thống đám mây thông qua việc sử dụng các công cụ để cấp phát và quản lý tài nguyên bằng mã nguồn.

Về định hướng phát triển, hệ thống sẽ tiếp tục hoàn thiện hơn nữa các tính năng phía Quản trị viên và yêu cầu gửi email chưa đạt được trong quá trình kiểm thử, tích

hợp các quy trình tích hợp và triển khai tự động khi mã nguồn được đẩy lên các kho chứa của Git bằng công cụ Atlantis, từ đó các nhà phát triển không cần phải chạy các lệnh thủ công để đóng gói và triển khai mã nguồn lên hệ thống đám mây của AWS. Ngoài ra, hệ thống cũng cần phát triển khả năng tích hợp với các hệ quản trị dữ liệu người dùng vốn có của doanh nghiệp như Microsoft Active Directory, phát triển các tính năng xác thực đa yếu tố khi đăng nhập và chống tấn công bằng các mã captcha. Cuối cùng, để gia tăng tính bảo mật của hệ thống, hệ thống trong tương lai sẽ phát triển thêm việc sử dụng mã xác thực dạng “phantom token” để tránh lộ lọt thông tin thông qua mã xác thực dạng JWT.

TÀI LIỆU THAM KHẢO

- [1] Fred B. Schneider, *CS 513 System Security -- Something You Know, Have, or Are*, [Trực tuyến]:
<https://www.cs.cornell.edu/courses/cs513/2005fa/NNLauthPeople.html>.
- [2] Ado Kukic, *The Definitive Guide To Single Sign On (SSO)*, [Trực tuyến]:
<https://auth0.com/resources/whitepapers/definitive-guide-to-single-sign-on>.
- [3] IBM, *What is serverless?*, IBM, [Trực tuyến]:
<https://www.ibm.com/topics/serverless>.
- [4] Peter Sbarski, *Serverless Architectures on AWS, 1st Edition*, Shelter Island, New York: Manning Publications, 2017.
- [5] Scott Winkler, *Terraform In Action, 1st Edition*, Shelter Island, New York: Manning Publications, 2021.
- [6] HashiCorp, *What is Terraform?*, [Trực tuyến]:
<https://developer.hashicorp.com/terraform/intro>.
- [7] HashiCorp, *What is Infrastructure as Code with Terraform?*, [Trực tuyến]:
<https://developer.hashicorp.com/terraform/tutorials/aws-get-started/infrastructure-as-code>.
- [8] Yevgeniy Brikman, *Terraform: Up and Running, 3rd Edition*, Sebastopol, California: O'Reilly Media, Inc., 2022.